NEURAL NETWORKS / DAY 5

Sequence models and attention

Forecasting, recurrent networks, attention, uncertainty, and a review of the course

Full day · 09:00–17:00 | 7 parts | 3 lab blocks | Final course review

Today's plan

09:00	Welcome · Part 1: time-series fundamentals	Lecture
09:45	Part 2: splits, baselines and metrics	Lecture
10:30	Break	Break
10:45	Part 3: recurrent networks	Lecture
11:30	Lab 1A: shampoo warm-up, climate baselines	Lab
12:30	Lunch	Lunch
13:30	Lab 1B: climate LSTM against baselines	Lab
14:15	Part 4: convolutions and attention	Lecture
14:50	Part 5: transformers, pretrained models	Lecture
15:20	Break	Break
15:35	Part 6: uncertainty and anomalies	Lecture
16:00	Lab 2: sequence anomalies or retail	Lab
16:35	Part 7: course review, exit check	Discussion

LEARNING OUTCOMES

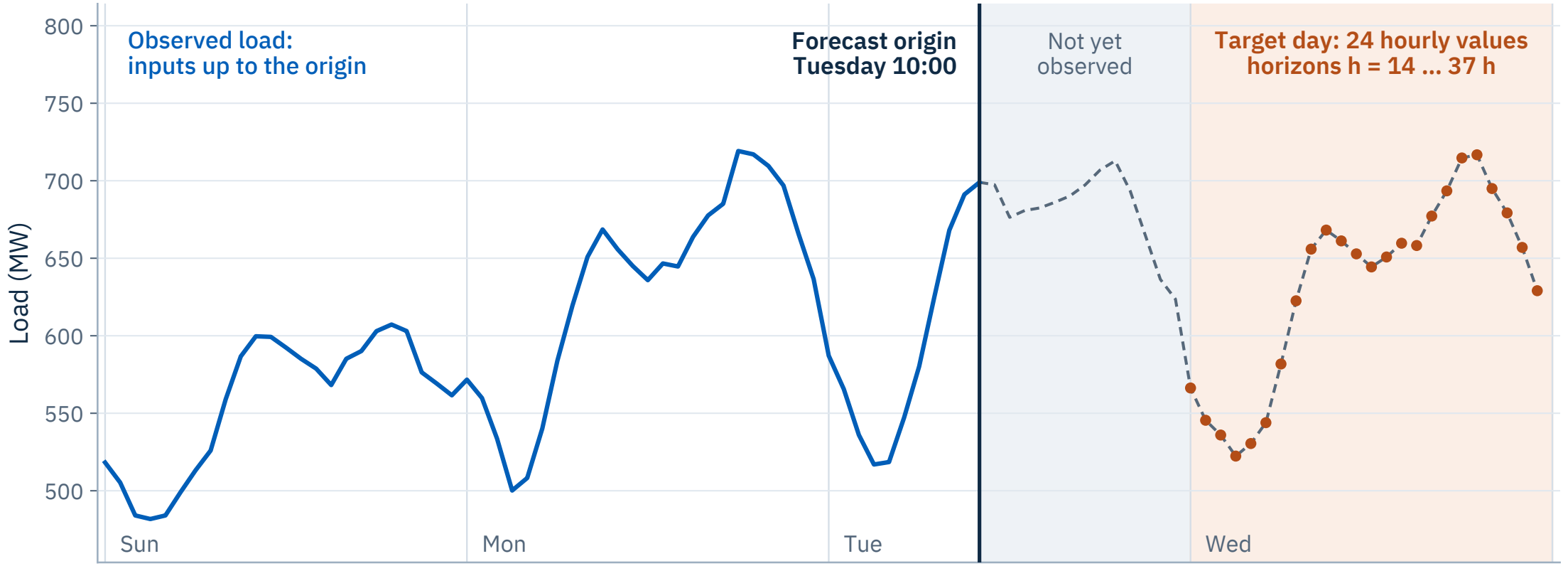
What you will be able to do

By the end of the day you will be able to:

1. Build forecast windows whose inputs are known at the origin, and count the samples.
2. Evaluate a forecast with a time-respecting split, strong baselines and per-horizon metrics.
3. Explain how RNN, LSTM and GRU states update, and check their tensor shapes in PyTorch.
4. Compute a small attention example, apply a causal mask and describe a transformer block.
5. Produce and check quantile forecasts and reconstruction-based anomaly scores.
6. Choose a model family for a new energy problem and defend its evaluation design.

A day-ahead forecast is fixed hours before its target

Next-day hourly load is due at 10:00, so every target lies 14 to 37 hours ahead.



Synthetic hourly load (seed 11) with trend, daily and weekly cycles, and correlated noise. Submission time is hypothetical.

PRACTICE

4 min · Pairs

A forecast is indexed by its origin

At 12:00 you must predict air temperature at 18:00 today.

1. Which measurements are already known, and what is the horizon?
2. Can you use tomorrow's observed maximum temperature?
3. How do known calendar features differ from future sensor readings?

PART 1 OF 7

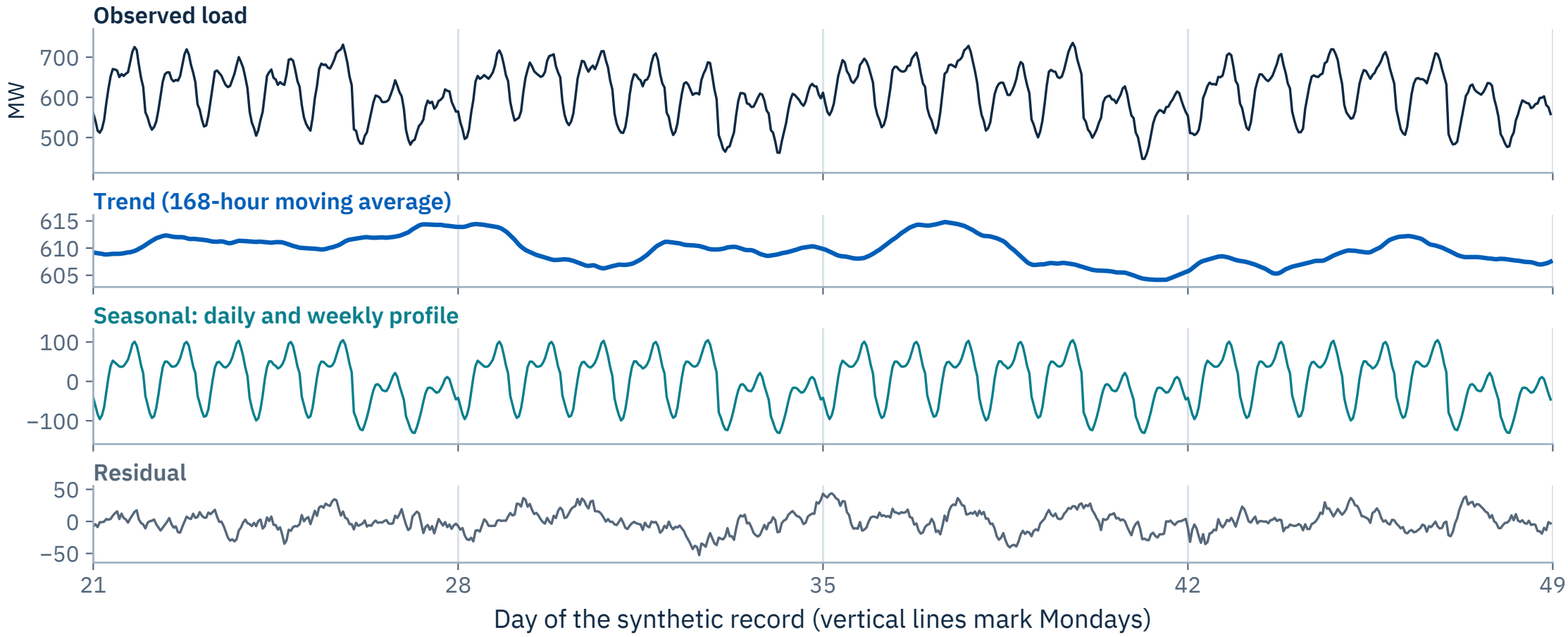
01

Time-series fundamentals

Structure in time, and what a forecast may use.

- Trend, daily and weekly cycles, and noise
- Autocorrelation, stationarity and differencing
- Origin, lookback, horizon and stride
- Sequence task types and covariates

An hourly load series mixes trend, cycles and noise



Synthetic hourly load (seed 11), weeks 4–7. Classical additive decomposition with a 168-hour period.

Classical decomposition takes four steps

The same recipe works for any period: 24 for a daily cycle, 168 for a weekly one.

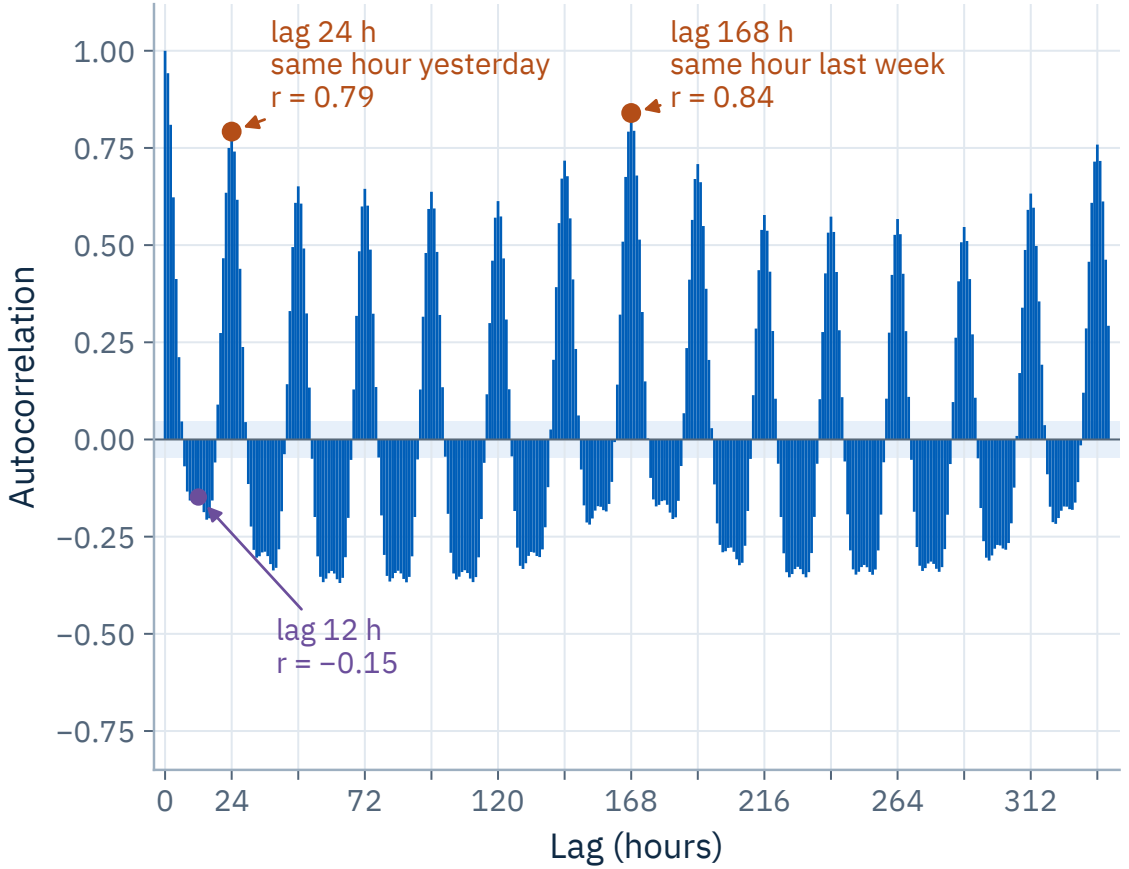
1. **Trend**: a centred 168-hour moving average smooths out the weekly cycle
2. **Detrend**: subtract the trend, leaving the cycles and the noise
3. **Seasonal profile**: average each of the 168 hours of the week, then centre it
4. **Residual**: whatever the trend and the seasonal profile leave unexplained

◆ A centred average reads 84 hours ahead: fine for description, leakage if used as a model input.

Autocorrelation reveals daily and weekly memory

$$r_k = \frac{\sum_t (y_t - \bar{y})(y_{t+k} - \bar{y})}{\sum_t (y_t - \bar{y})^2}$$

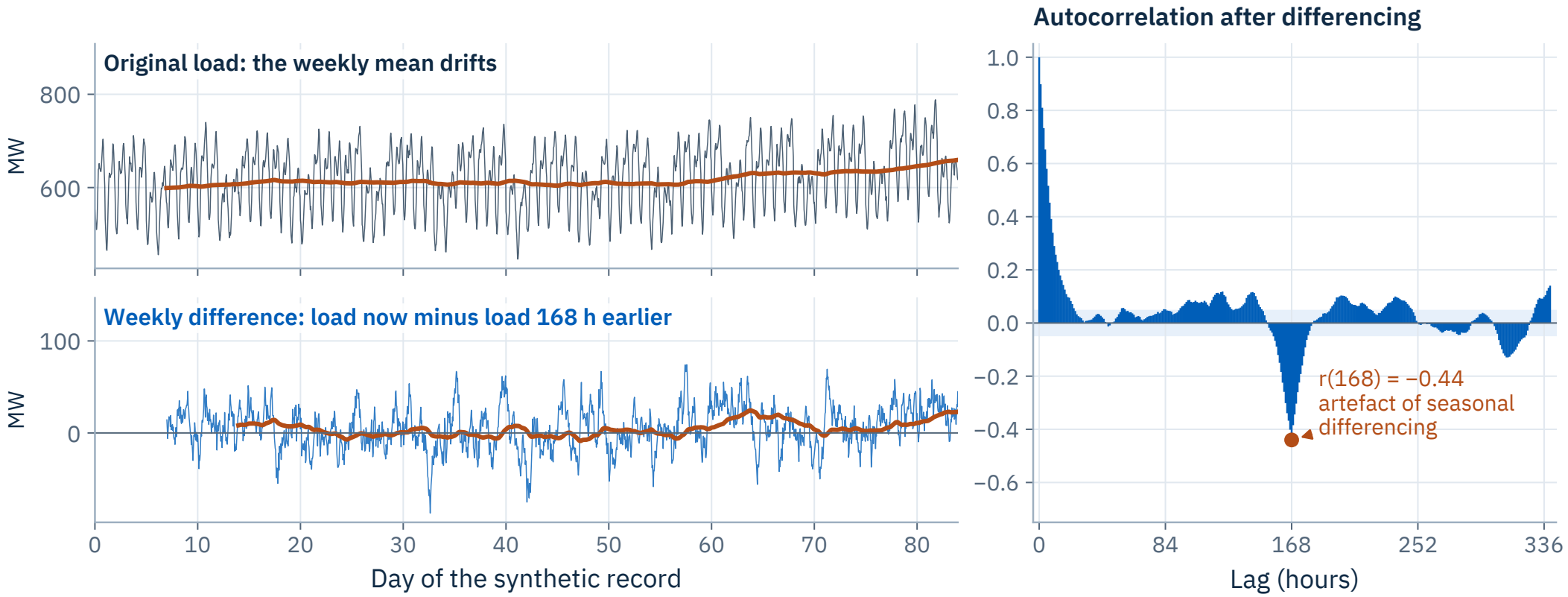
- r_k correlates the series with itself k steps later.
- Peaks at 24 and 168 hours: yesterday and last week.
- Negative at 12 hours: the opposite phase of the day.



Synthetic hourly load (seed 11, 2,016 hours). Band: approximate 95% range for white noise.

Differencing removes trend and seasonality

Weakly stationary: mean, variance and autocorrelation stay the same over time.



Synthetic hourly load (seed 11). Orange: 168-hour rolling means. Right: autocorrelation of the weekly difference.

PRACTICE

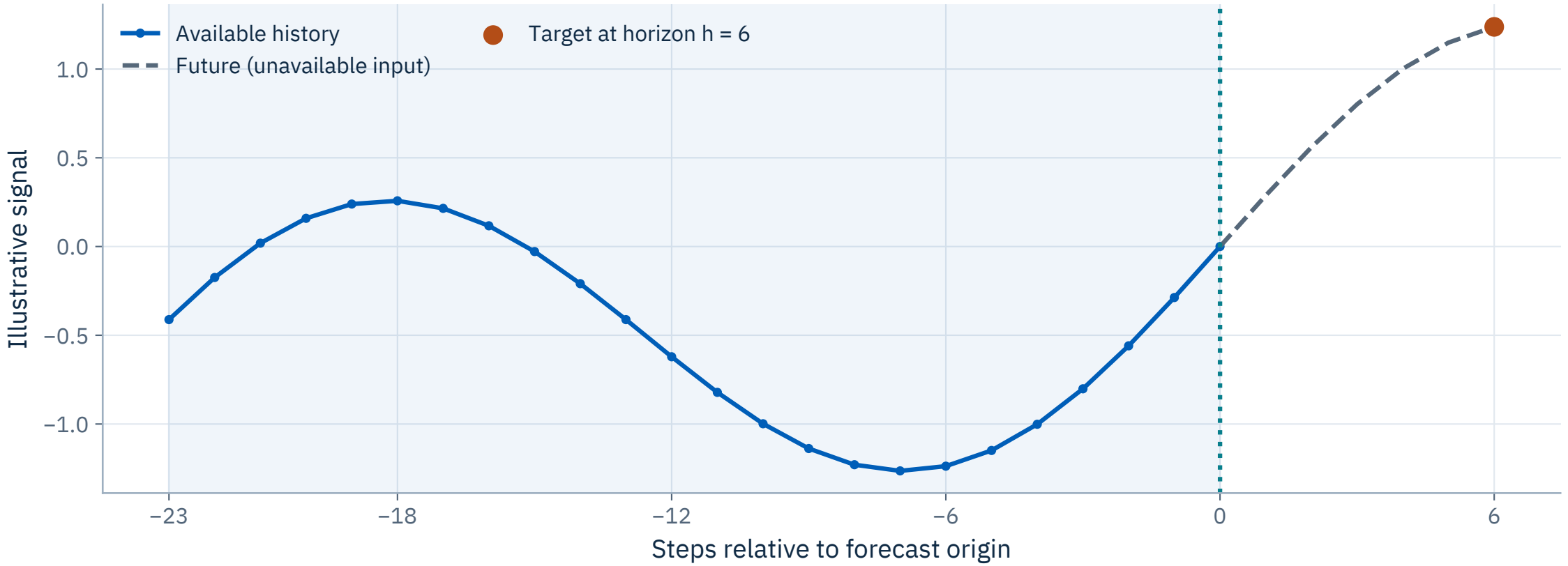
4 min · Pairs

Read the autocorrelation before choosing a lookback

For the hourly load: r(1) = 0.94, r(12) = −0.15, r(24) = 0.79, r(168) = 0.84.

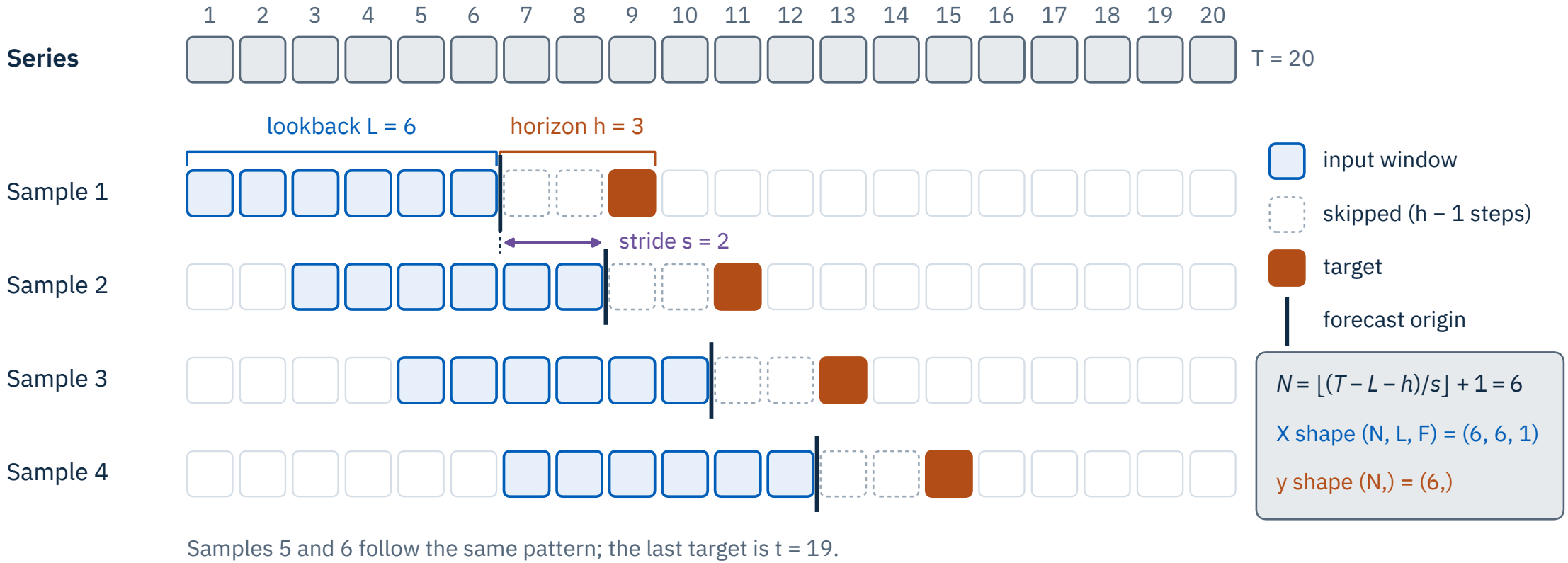
1. Which seasonal-naive forecast does this suggest, and why not lag 24?
2. Why is r(12) negative?
3. For the 10:00 day-ahead case, is lag 24 even available for 18:00 tomorrow?

A window fixes origin, lookback and horizon



Schematic single-origin forecast: lookback 24 observations, horizon 6 steps. Inputs end at the origin; the target lies h steps later.

Sliding the origin turns one series into many samples



Samples 5 and 6 follow the same pattern; the last target is t = 19.

Schematic with exact indexing: T = 20, L = 6, h = 3 and s = 2 give N = 6 samples.

A window builder in a few lines of NumPy

```python
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view

def make_windows(series, L, h, s=1):
    """Inputs end at each origin; the target is h steps later."""
    X = sliding_window_view(series[:len(series) - h], L)[::s]
    y = series[L - 1 + h::s][:len(X)]
    return X[..., None], y            # (N, L, 1) and (N,)

X, y = make_windows(np.arange(1.0, 21.0), L=6, h=3, s=2)
print(X.shape, y.shape)
print(X[0, :, 0], y[0])
```

OUTPUT

```
(6, 6, 1) (6,)
[1. 2. 3. 4. 5. 6.] 9.0
```

Build windows after the chronological split, separately for each asset.

Known and unknown covariates differ at the origin

✓ Known in advance

- Calendar: hour, weekday, holidays
- Planned maintenance and set points
- Weather forecasts issued before the origin

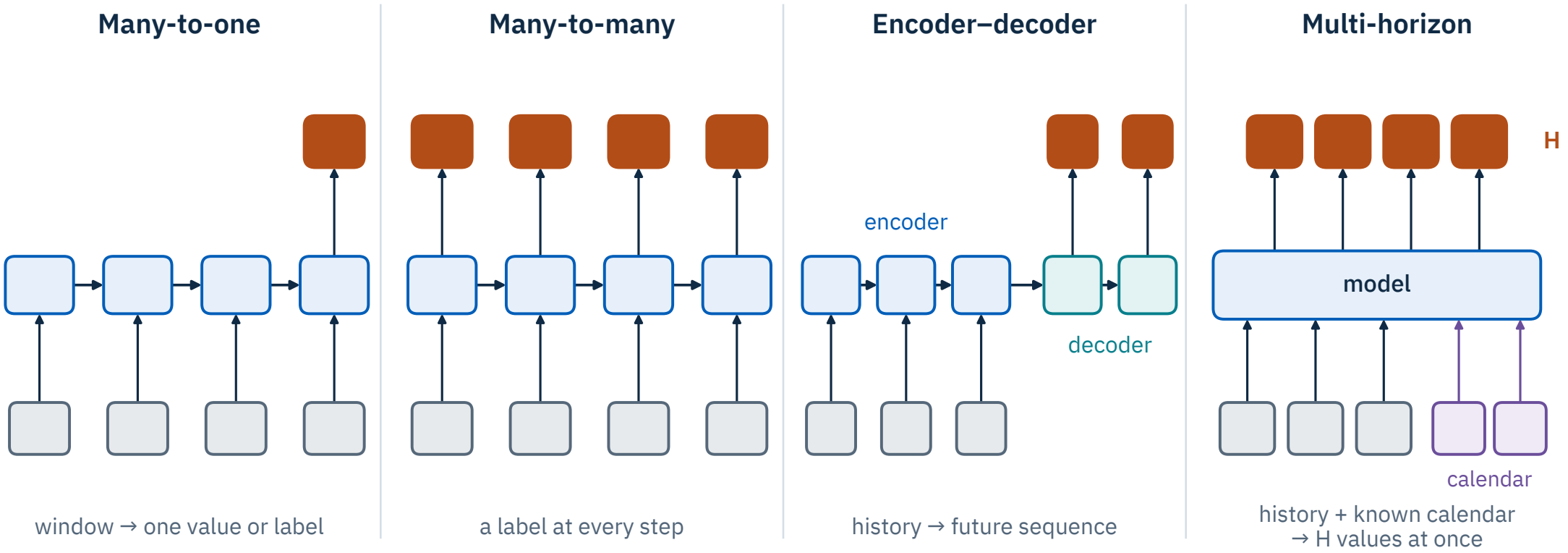
vs

✗ Unknown at the origin

- Future load, production or pressure
- Realized weather after the origin
- Values that arrive after the origin

◆ Check when each value became available, not only when it was measured.

Sequence tasks have different outputs



Schematic. Grey: inputs; blue and teal: learned states; orange: outputs; purple: covariates known in advance.

PRACTICE

5 min · Individual

Count the windows before you train

The climate lab uses hourly data, lookback L = 72 and horizon h = 12.

1. How many windows does a 1,000-hour segment give with stride 1? With stride 6?
2. The training segment has 49,064 hours. How many training windows?
3. Why does the notebook build windows separately inside each split?

PART 2 OF 7

02

Splits, baselines and metrics

A forecast is only as credible as its evaluation.

- Splits that respect target intervals
- Rolling origins and strong baselines
- Metrics and their pitfalls
- Multi-step strategies and horizons

Time needs a different split from rows or wells

Random rows

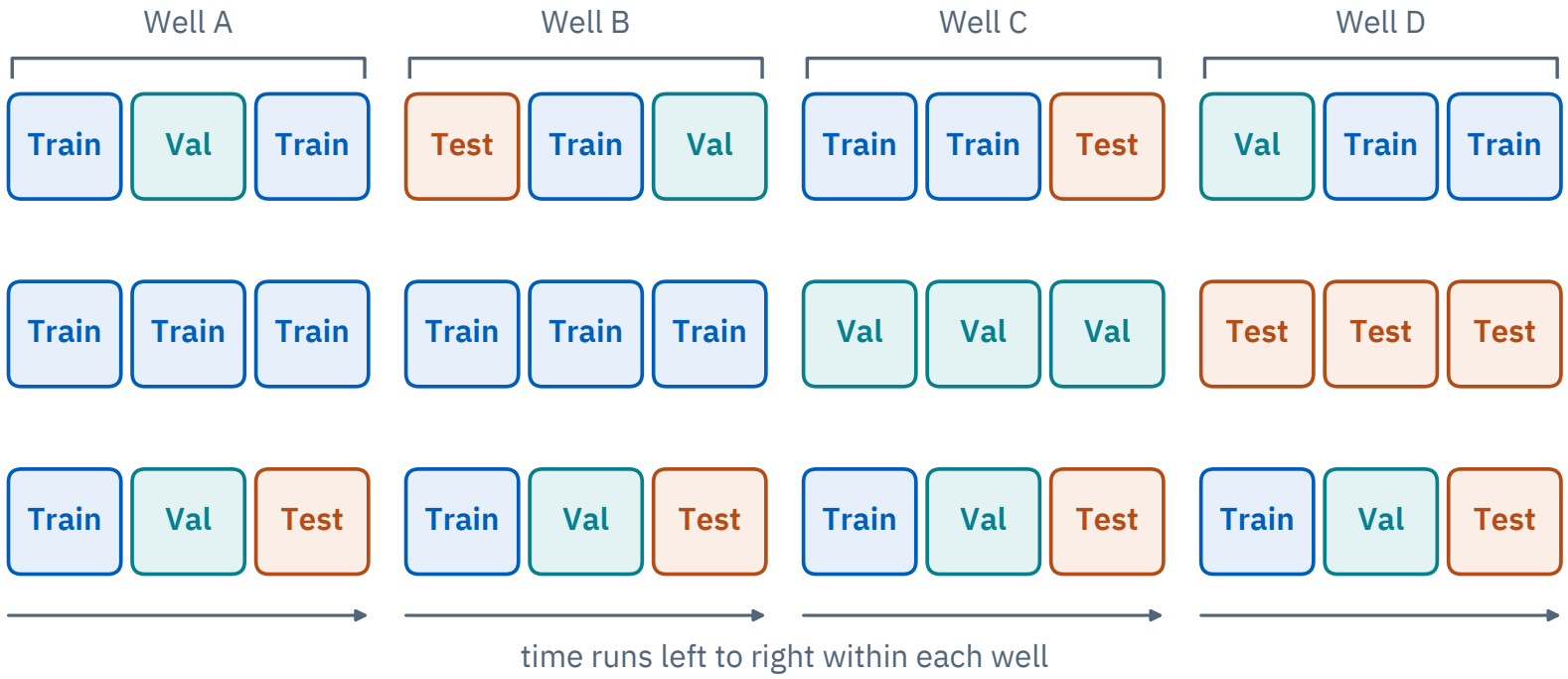
tests new rows from the same wells

Grouped by well

tests wells never seen in training

Ordered in time

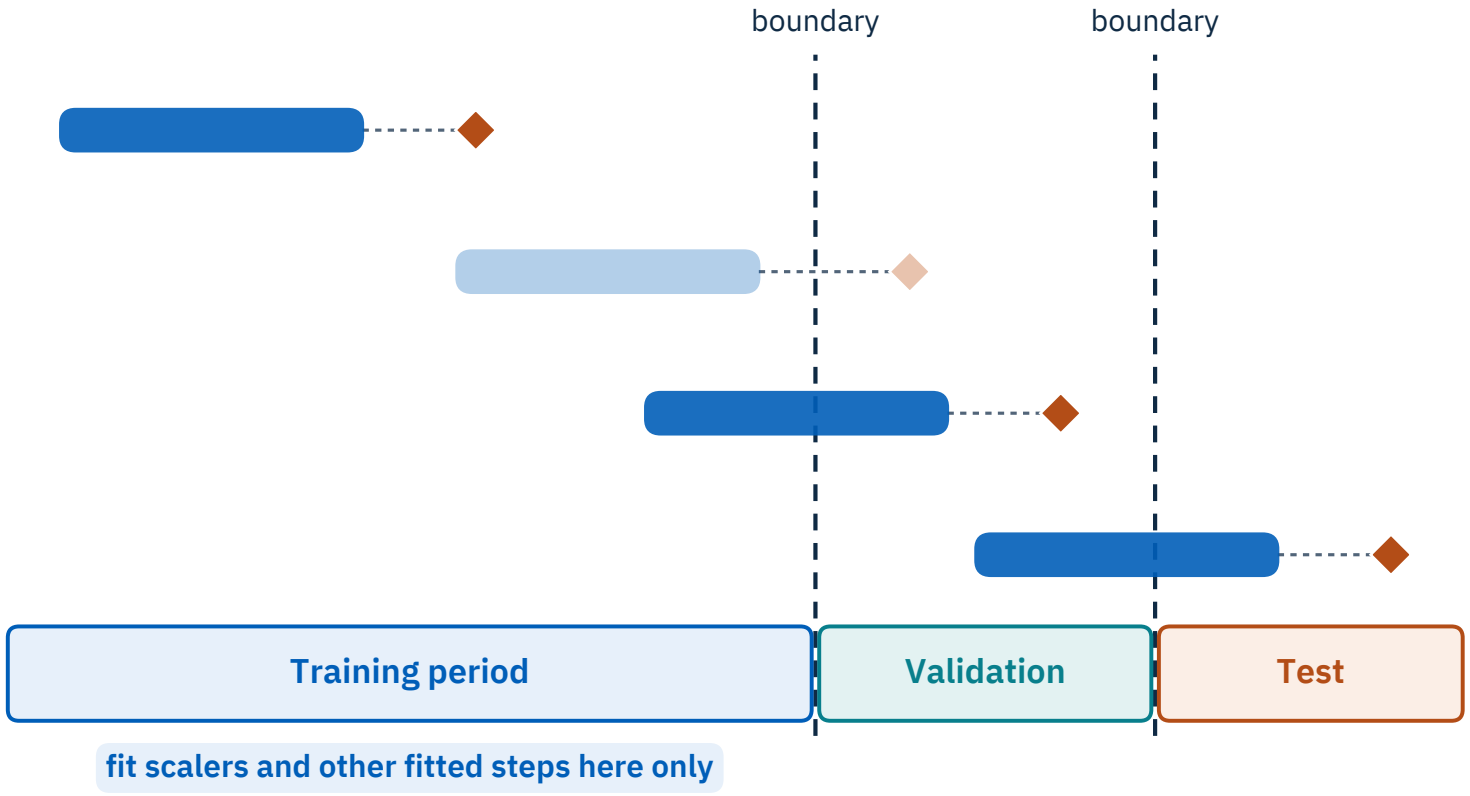
tests the later period of every well



Schematic partition assignments: a time split holds out the later period of every well. The machine learning course covers these splits in more depth.

Targets, not inputs, define the split boundary

- **Training window**
  inputs and target in training
- **Target after the boundary**
  purge it from training
- **Validation window**
  may read earlier, observed history
- **Test window**
  scored once, at the end



Schematic. Bars are input windows, diamonds are targets. The faded window is removed because its target falls after the boundary.

Four rules keep time splits honest

Fit on the past only

- Scalers, imputers, encoders, thresholds
- Estimated on training rows, then frozen

Purge crossing targets

- No training target after the boundary
- Embargo gaps for multi-step labels

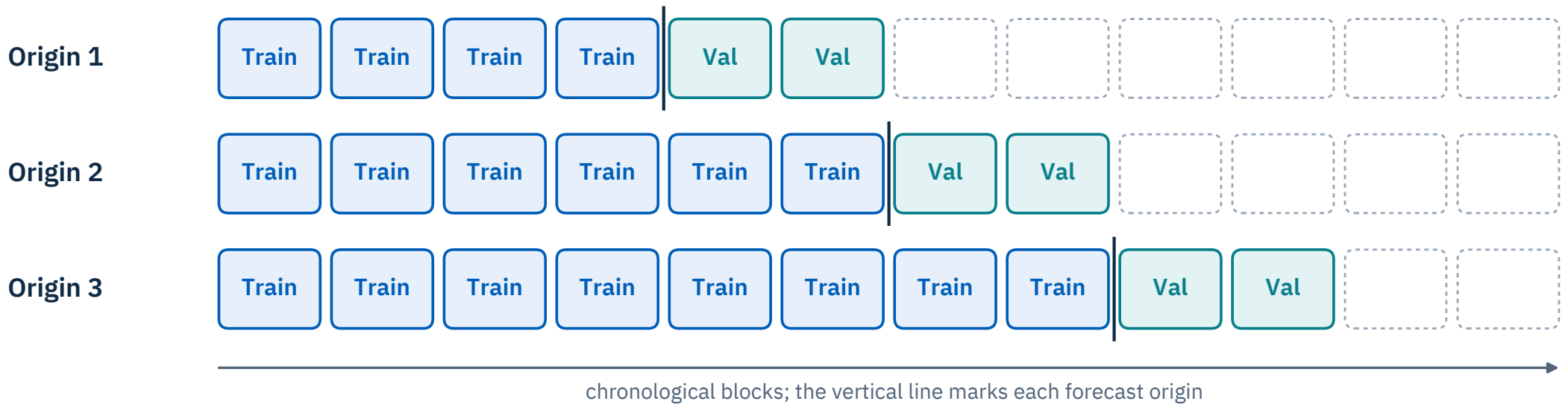
Allow observed history

- Validation inputs may start earlier
- That is what deployment sees

Keep assets separate

- Windows never join two wells or feeders
- Reset recurrent state between assets

Rolling-origin evaluation repeats the real question



Schematic expanding-window evaluation. Each fold forecasts from information available at its origin; keep a final test period outside all folds.

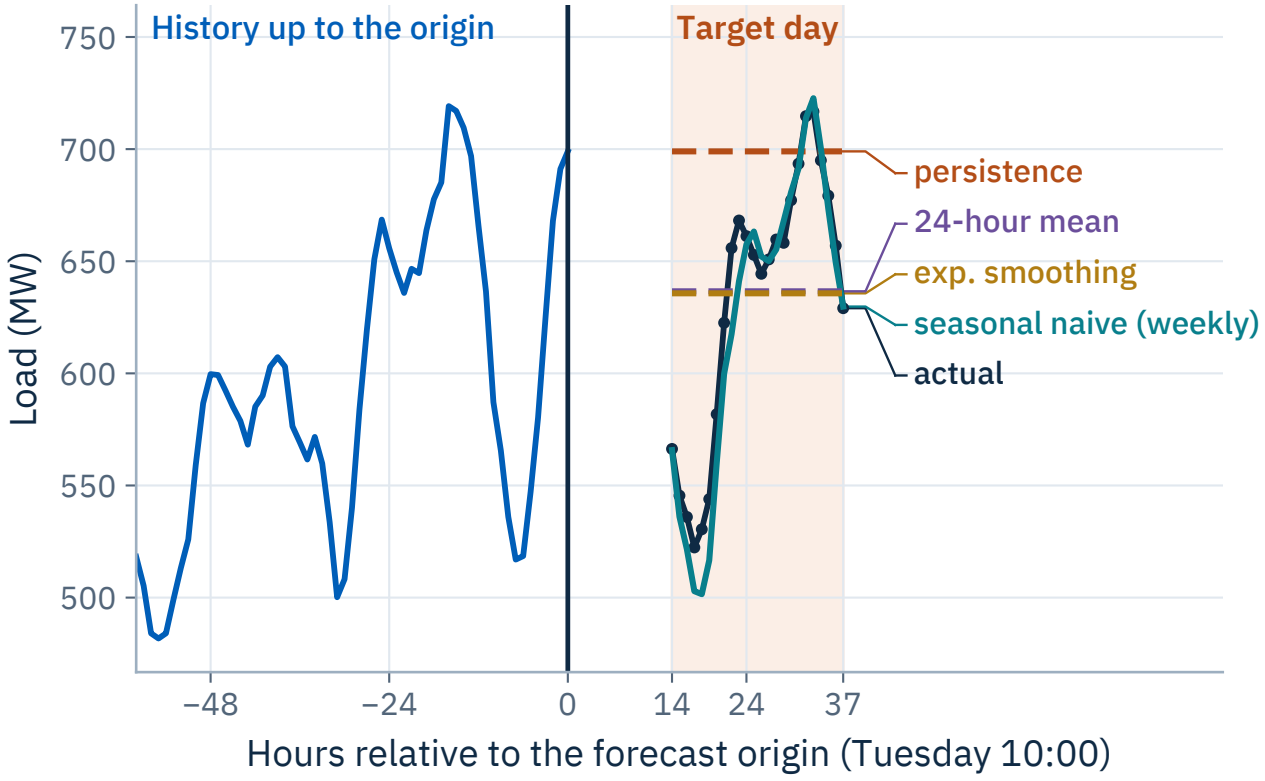
Strong baselines can be very simple

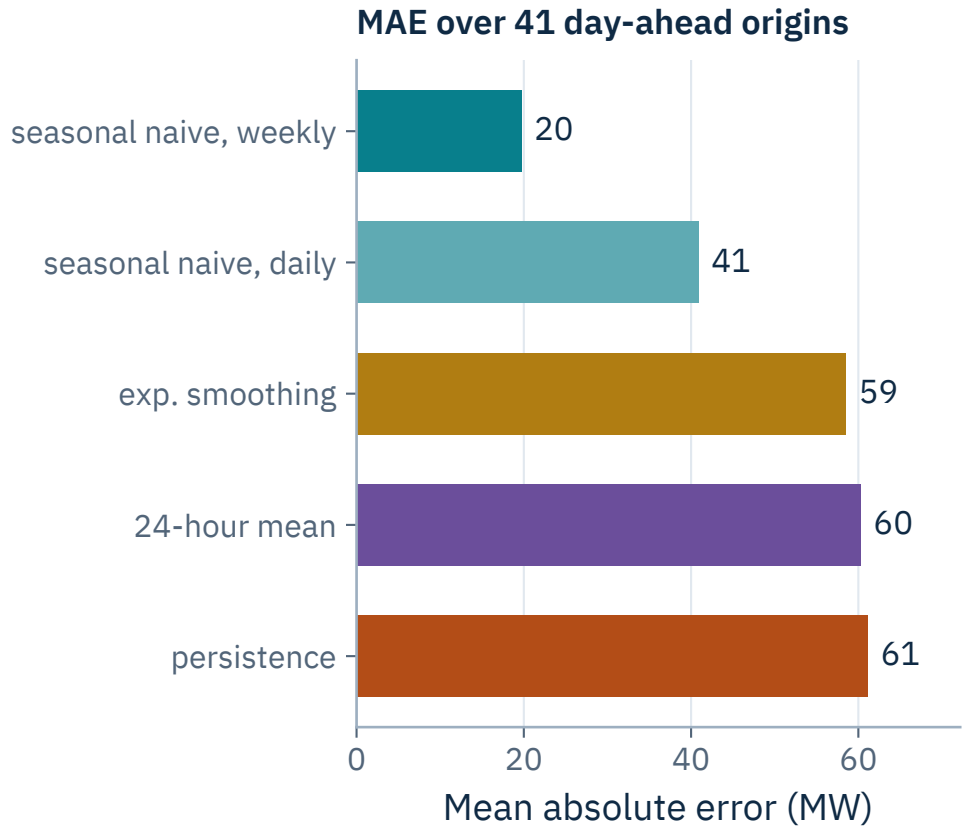
Baseline	Forecast of the value h steps ahead	Informative when
Persistence	The last observed value	Slowly changing signals
Seasonal naive	Same hour, one season earlier	Repeating daily or weekly cycles
Moving average	Mean of the last k values	A noisy level without cycles
Exponential smoothing	A level updated with weight α	A level that drifts slowly
Lag model	Linear or boosted on lags and calendar	A tabular benchmark

Score every model on the same origins, targets and horizons.

◆ A model that cannot beat these has not yet earned its complexity.

Seasonal naive wins at day-ahead horizons





Synthetic hourly load (seed 11): 41 origins at 10:00 on days 42–82, targets 14–37 hours ahead; exponential smoothing α = 0.2.

PRACTICE

3 min · Individual

Exponential smoothing by hand

Simple exponential smoothing with $\alpha = 0.5$ starts from a level $\ell_0 = 100$ MW.

1. The next two observations are 110 and 90 MW. Compute ℓ_1 and ℓ_2.
2. What is the forecast for 1 hour ahead, and for 24 hours ahead?
3. What does the method become when α approaches 1?

Forecast metrics answer different questions

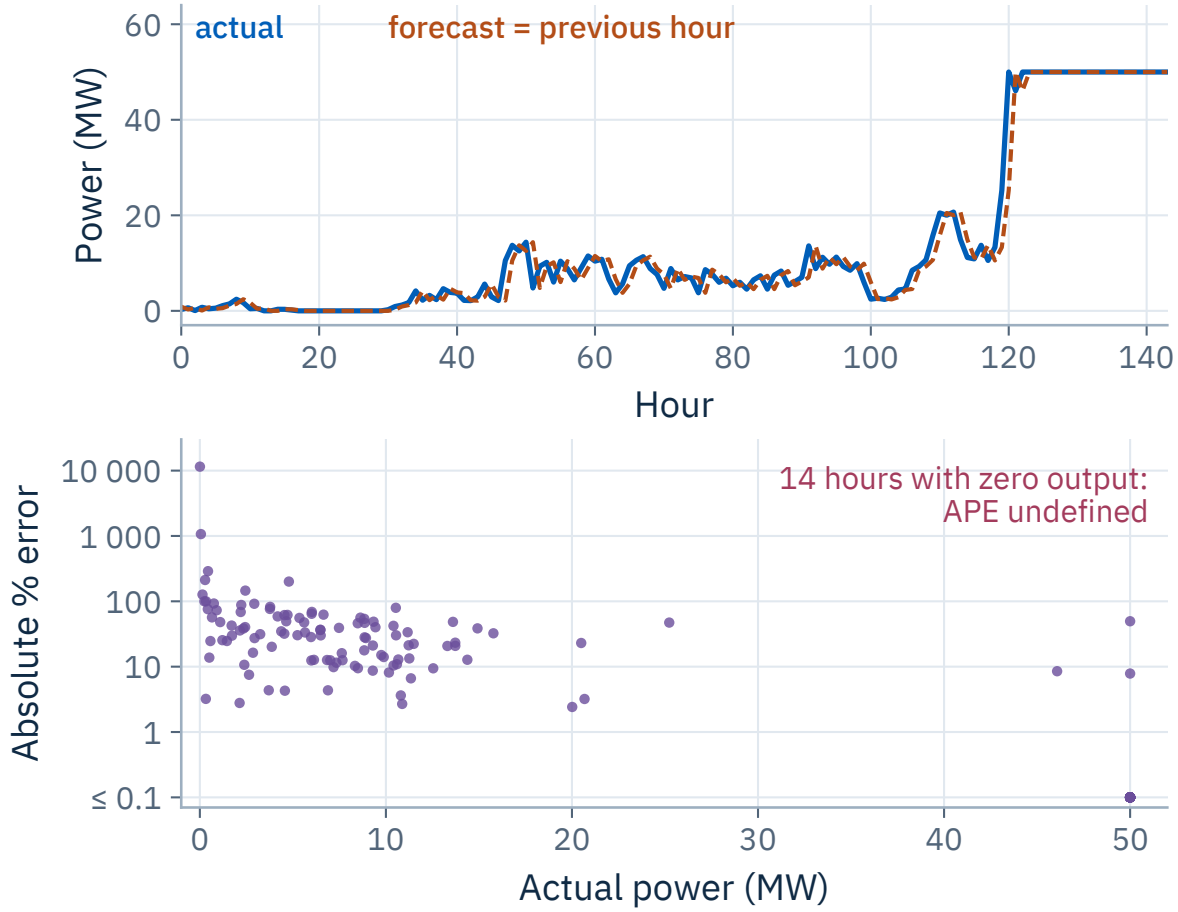
Metric	Definition	Watch out for
MAE	Mean absolute error	In target units; robust to single misses
RMSE	Root of the mean squared error	Dominated by the largest misses
MAPE	Mean of absolute error ÷ actual	Undefined at zero, explodes near zero
MASE	MAE ÷ in-sample naive MAE	Below 1 beats the naive method

Report errors by horizon and by asset, not only one average.

◆ Choose the metric before seeing any results, and use it for every model.

MAPE explodes when actual values approach zero

- One forecast: MAE = 1.7 MW, median APE = 23%.
- MAPE = 130%, driven by a few low-output hours.
- 14 zero-output hours cannot enter MAPE at all.



Synthetic 50 MW wind farm (seed 3), 144 hours; forecast = previous hour's output.

◆ For wind and solar, divide MAE by installed capacity instead.

MASE compares your error with a naive forecast

$$\mathrm{MASE} = \frac{\mathrm{MAE}}{\frac{1}{n-m}\sum_{t=m+1}^{n} |y_t - y_{t-m}|}$$

- The denominator uses the training period only.
- m = 1 scales by persistence; m = 24 or 168 by a seasonal naive.
- MASE below 1 means better than that naive method, on average.
- Scale-free, and defined when the series contains zeros.

◆ Example: MAE 18 MW against an in-sample weekly-naive MAE of 24 MW gives MASE = 0.75.

PRACTICE

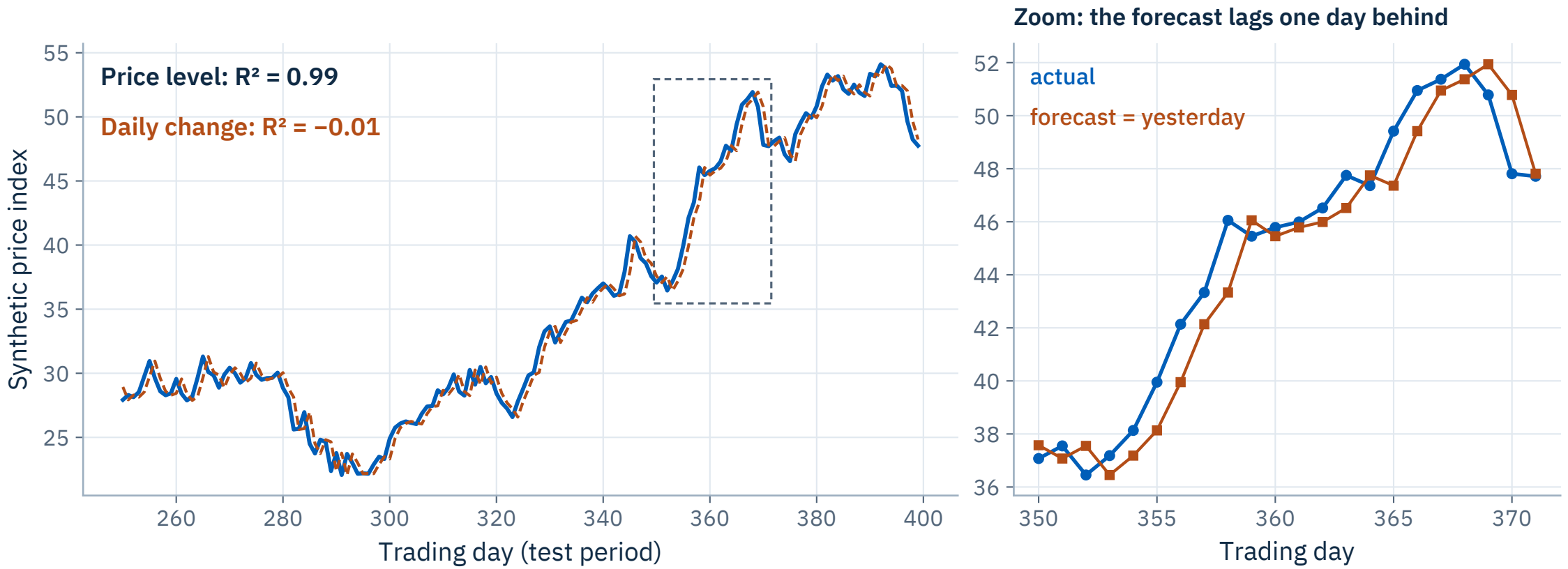
5 min · Pairs

Score one forecast four ways

Wind power (MW). Actual: 4, 2, 0.5, 6. Forecast: 3, 3, 1.5, 5. Training one-step naive MAE: 2.25 MW.

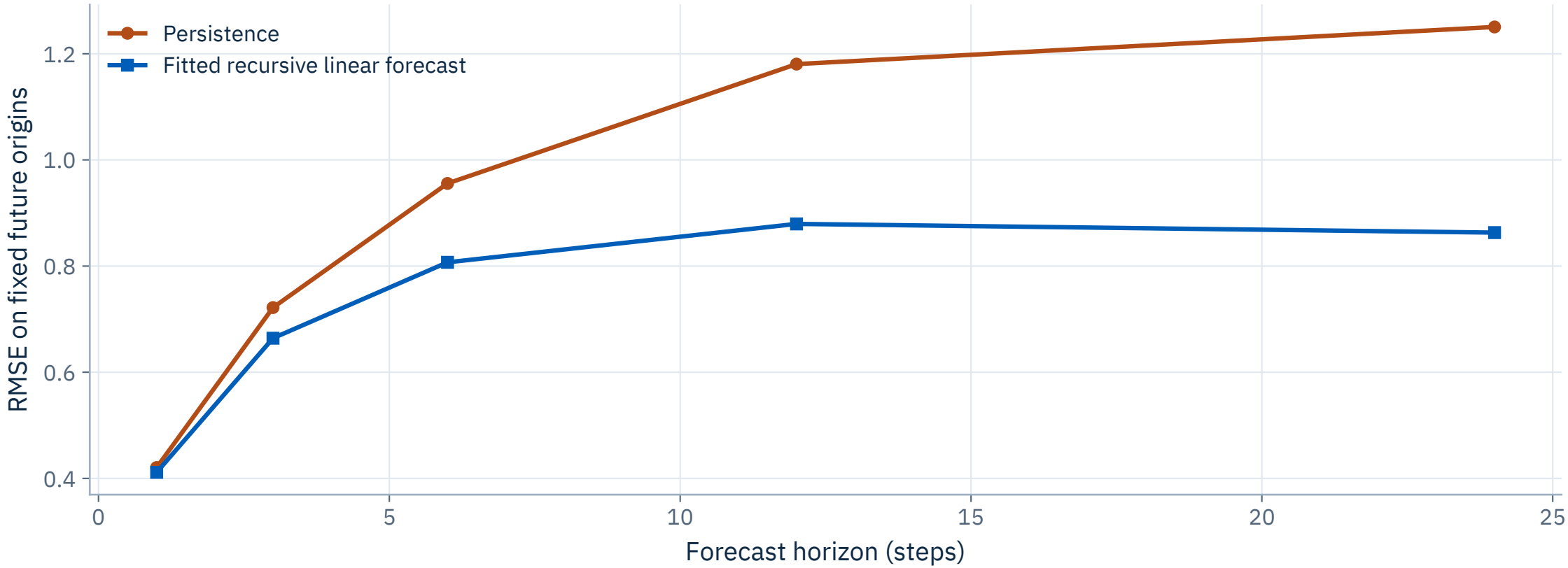
1. Compute MAE and RMSE.
2. Compute MAPE. Which hour dominates it?
3. Compute MASE and interpret it.

A price model can look perfect by copying yesterday



Synthetic Gaussian random walk (seed 21), test days 250–399; the forecast is simply the previous day's value.

Forecast error depends on horizon

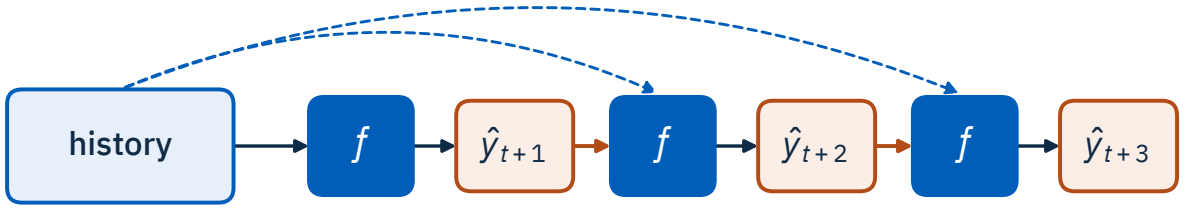


Simulated AR(1) series (coefficient 0.85, seed 42) scored on 320 fixed future origins; persistence and a recursive linear model fitted on earlier data.

Multi-step forecasts need a strategy

Recursive

one model, fed its own output

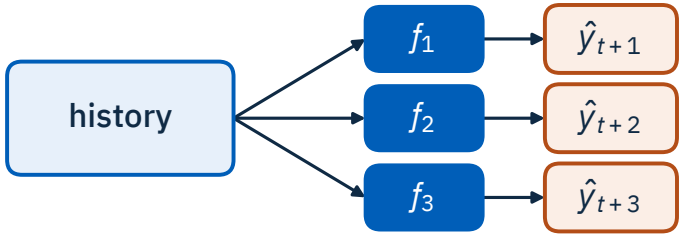


+ one model for any horizon

− errors feed forward and compound

Direct

one model per horizon

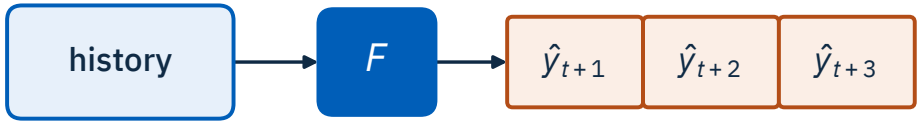


+ no feedback of errors

− H models; horizons trained apart

Multi-output

one model, a vector of outputs



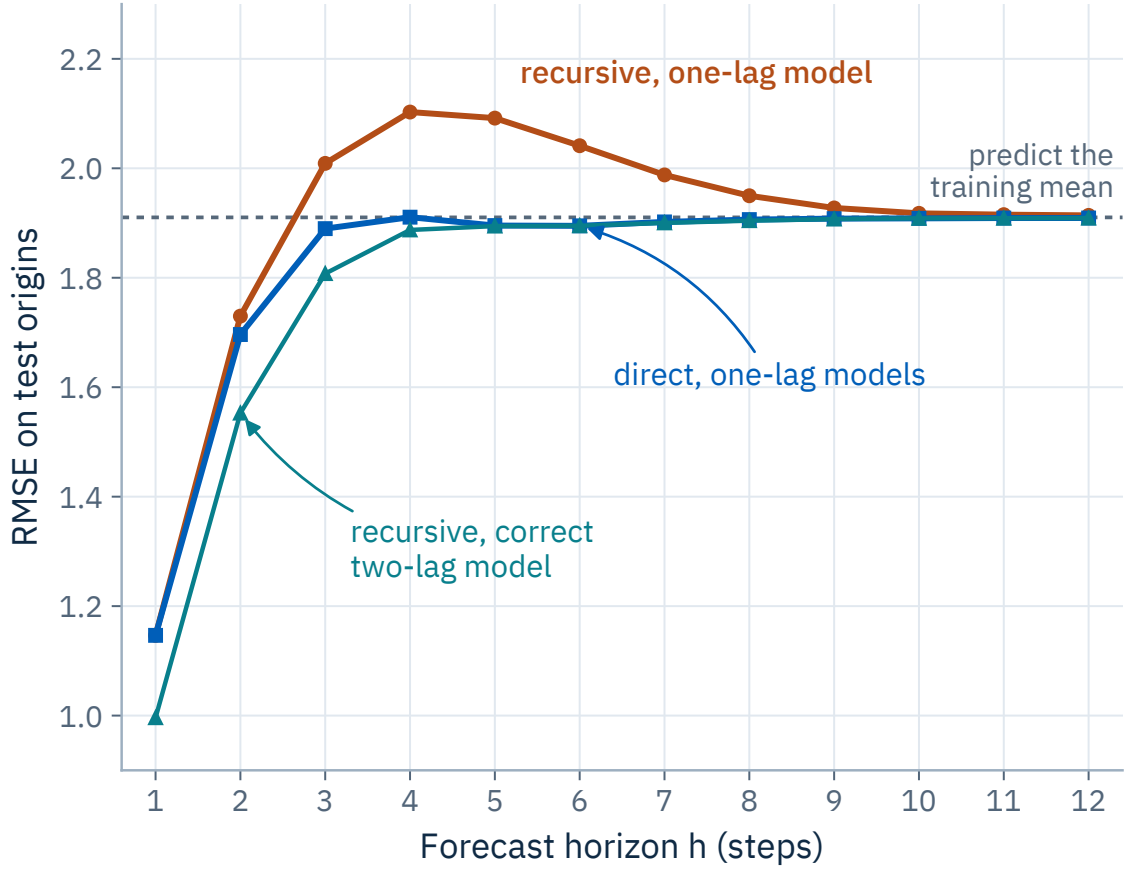
+ one model, horizons trained jointly

− horizon length fixed at design

Schematic of three strategies for horizons 1–3. Dashed arrows: the recursive model also re-reads the observed history.

A misspecified recursive model compounds its error

- True process uses two lags; the one-step model uses one.
- From h = 3, recursive errors exceed the mean forecast.
- Direct models level off at the mean-forecast error.
- With the correct two-lag model, recursion works well.



Simulated AR(2) process (seed 8); models fitted on the first 5,000 steps, scored on 1,986 later origins.

PRACTICE

5 min · Pairs

Design the evaluation for a wind forecast

A 50 MW wind farm needs hourly forecasts 1–48 hours ahead, issued every day at 06:00. You have two years of data.

1. Which baselines must a neural model beat?
2. Which metric would you report, broken down how?
3. Sketch the split and the rolling origins.

PART 3 OF 7

03

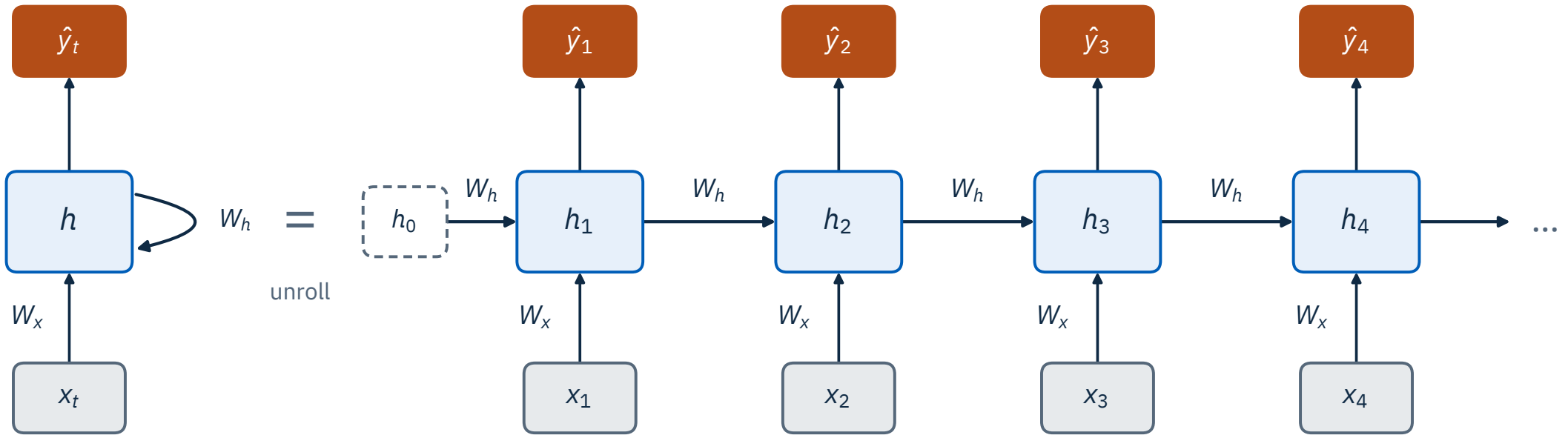
Recurrent networks

One shared update carries a state through time.

- The recurrent update and backpropagation through time
- Vanishing gradients and gated memory
- LSTM, GRU, stacked and bidirectional layers
- Shapes and state handling in PyTorch

One recurrent cell is reused at every step

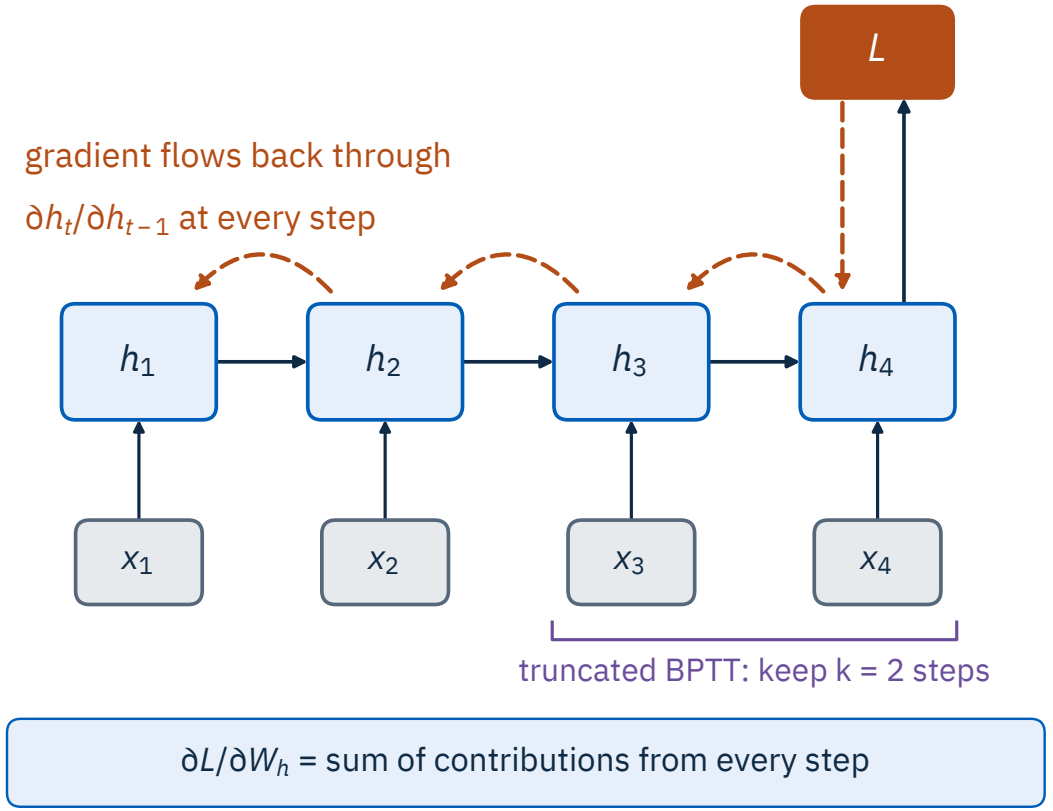
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$



Schematic. The same weights W_x, W_h and bias b are applied at every time step; h_0 is usually zeros.

Backpropagation through time follows the unrolled graph

- Unroll the shared recurrence over the training window.
- Gradients for W_h add up contributions from every step.
- Long products of derivatives can shrink or amplify.
- Truncated BPTT limits history to manage compute.

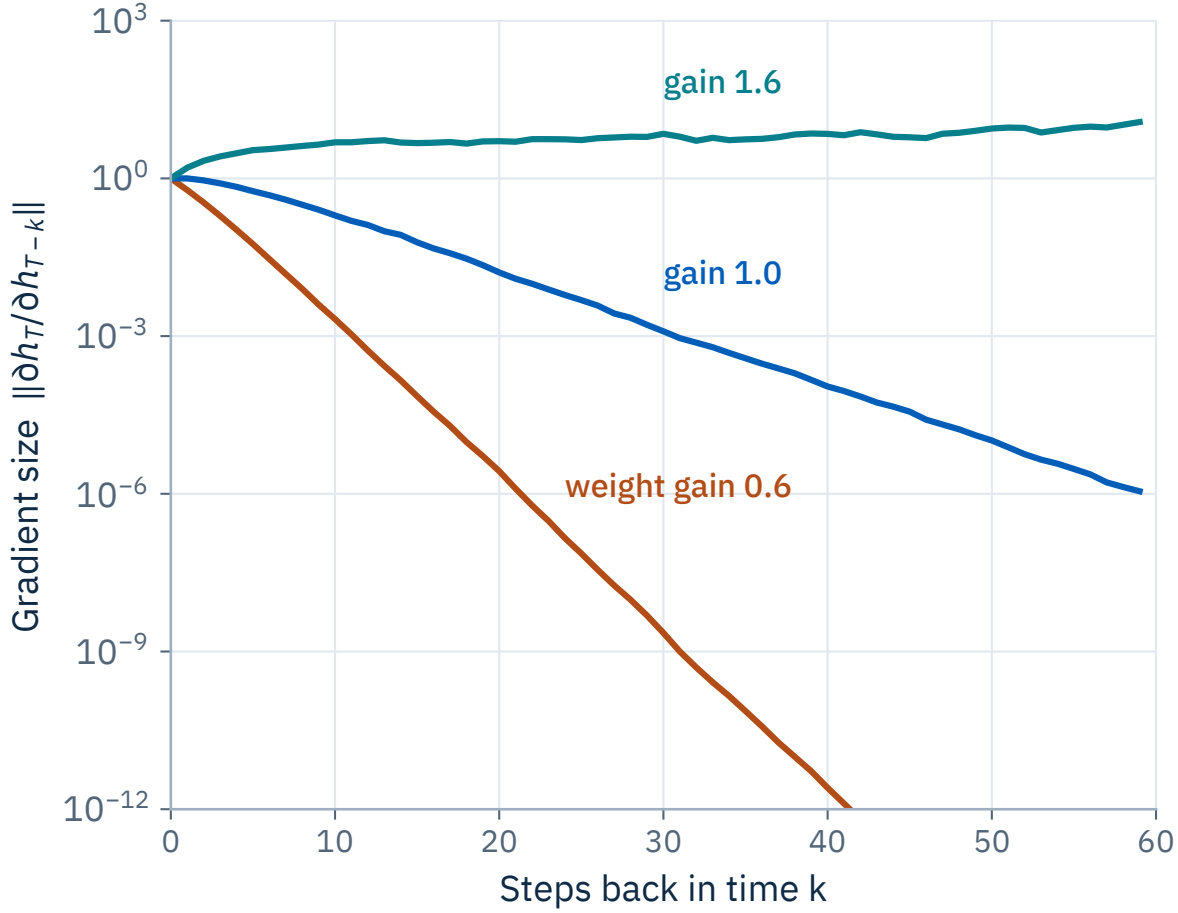


Schematic for a many-to-one RNN; dashed arrows carry gradients backwards through the recurrence.

Gradients shrink geometrically through time

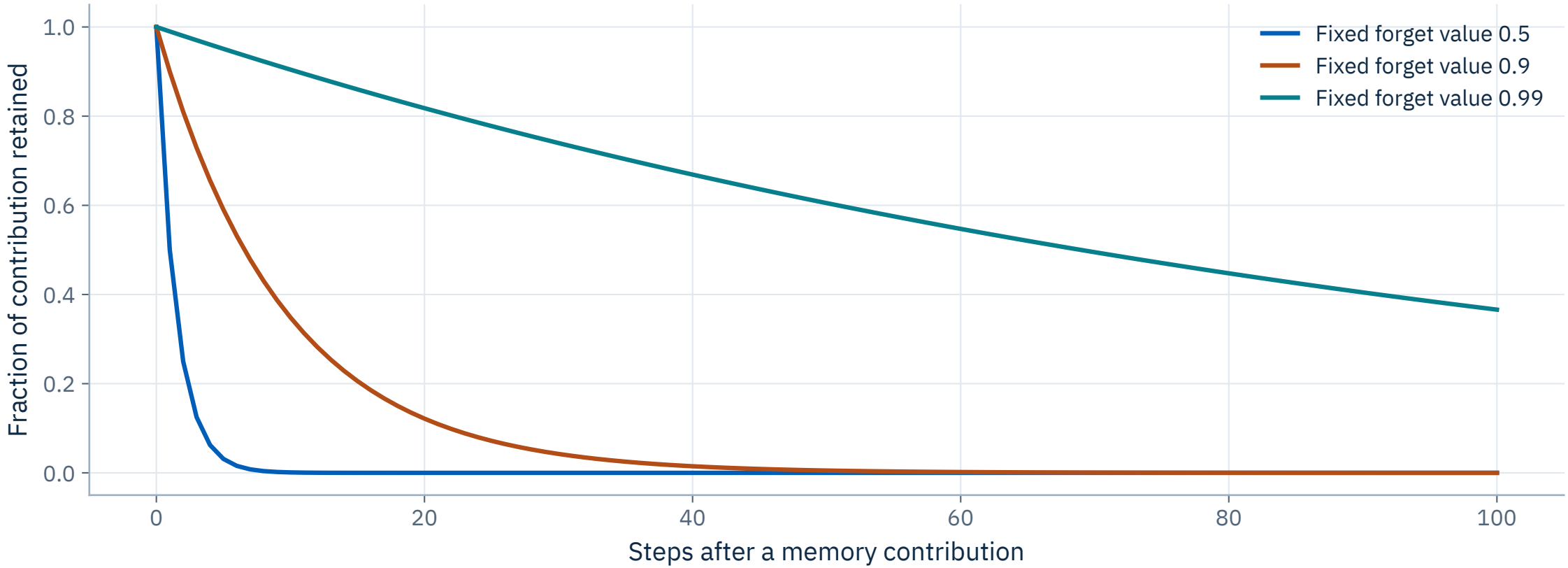
$$\frac{\partial h_T}{\partial h_{T-k}} = \prod_{j=T-k+1}^{T} \operatorname{diag}(1 - h_j^2)\, W_h$$

- Each step multiplies the gradient by a Jacobian.
- Gain 0.6: about 3×10^{-6} of the signal after 20 steps.
- Even gain 1.0 decays, because tanh slopes are below 1.



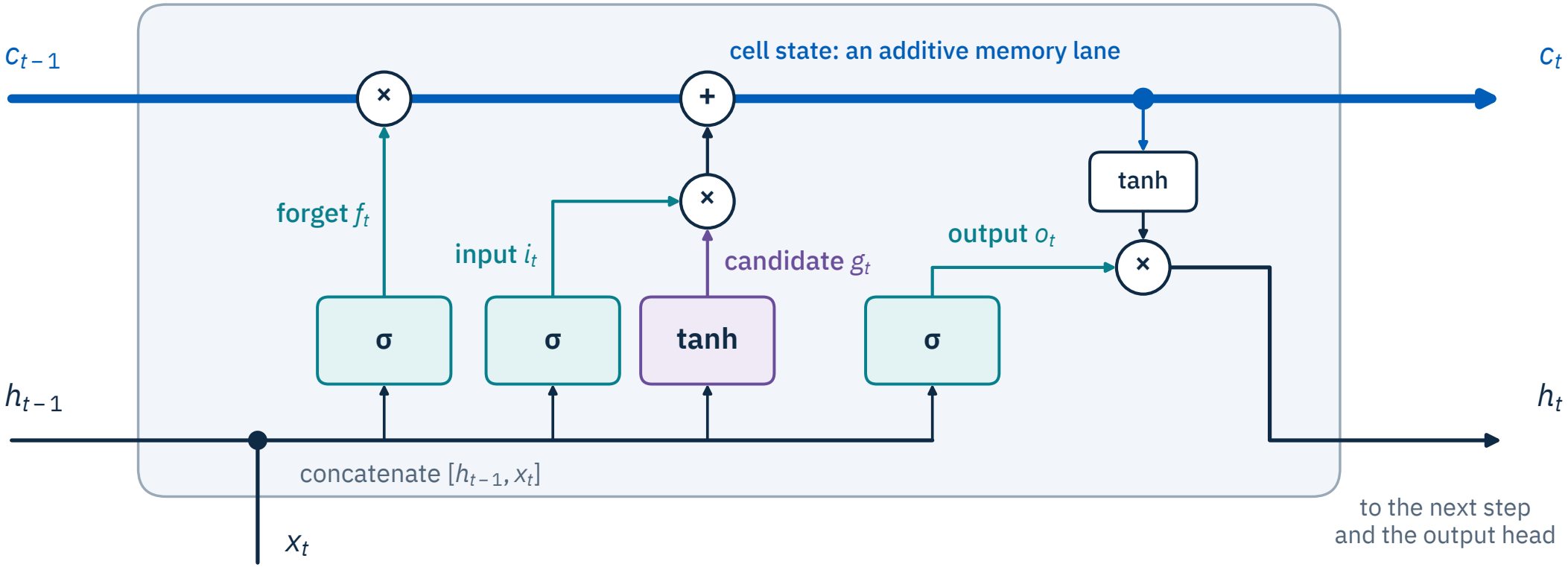
Computed for a 32-unit tanh RNN with random inputs (seed 4); median over 30 runs.

Gated memory offers a more direct path



Simplified retained-signal magnitude under fixed forget-gate values. Real LSTM gates depend on inputs and state.

An LSTM cell adds a protected memory lane



Schematic of one LSTM step. Teal: sigmoid gates; purple: candidate values; blue: cell state. × and + act elementwise.

Four gates define the LSTM update

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \qquad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$
$$g_t = \tanh(W_g[h_{t-1}, x_t] + b_g), \qquad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t)$$

- Gates use sigmoids (0 to 1); the candidate uses tanh (–1 to 1).
- ⊙ multiplies elementwise; c and h both have H values.

◆ PyTorch stacks the four weight blocks in the order i, f, g, o.

PRACTICE

4 min · Individual

A memory update by hand

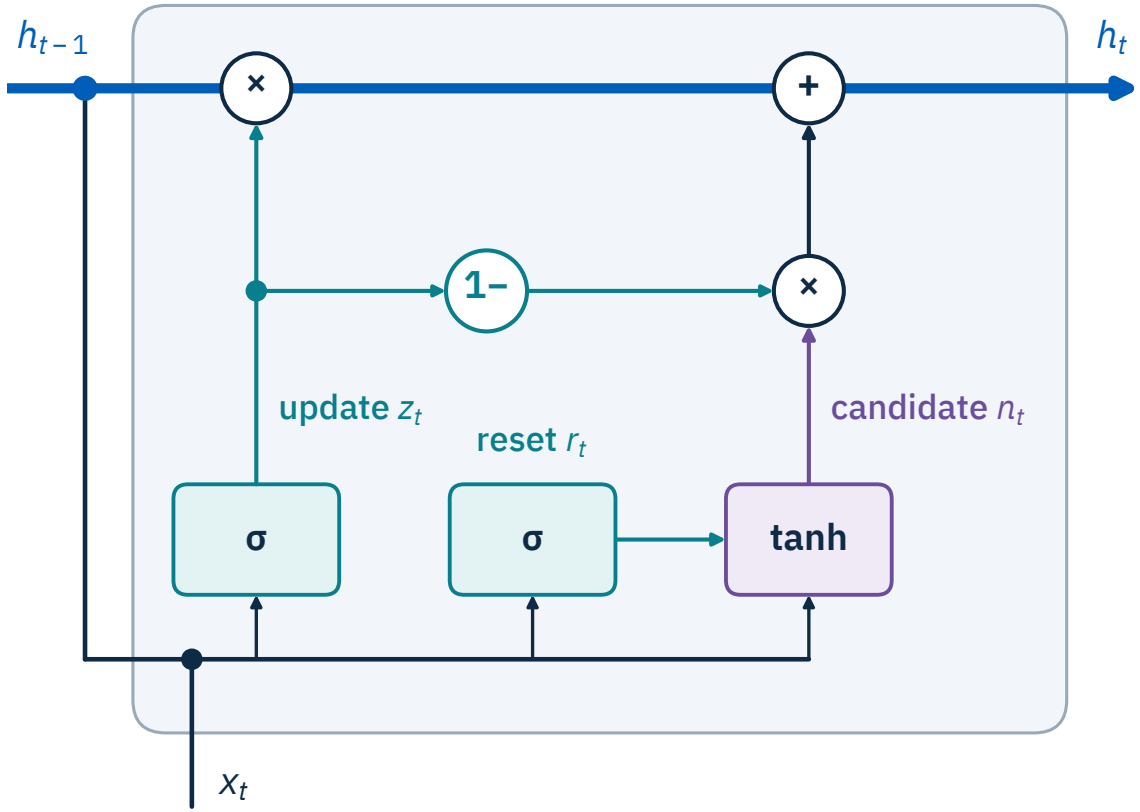
Previous cell value = 2. Forget gate = 0.9. Input gate = 0.2. Candidate = −1.

1. Calculate the new cell value.
2. With output gate 0.5, compute the hidden state.
3. With the forget gate fixed at 0.9 and no new input, what fraction of the memory remains after 10 steps?

A GRU blends old and new state with one gate

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$$

- Update gate *z* blends the candidate with the old state.
- Reset gate *r* scales the old state inside the candidate.
- No separate cell state; three weight blocks, not four.



Schematic of one GRU step in the PyTorch convention.

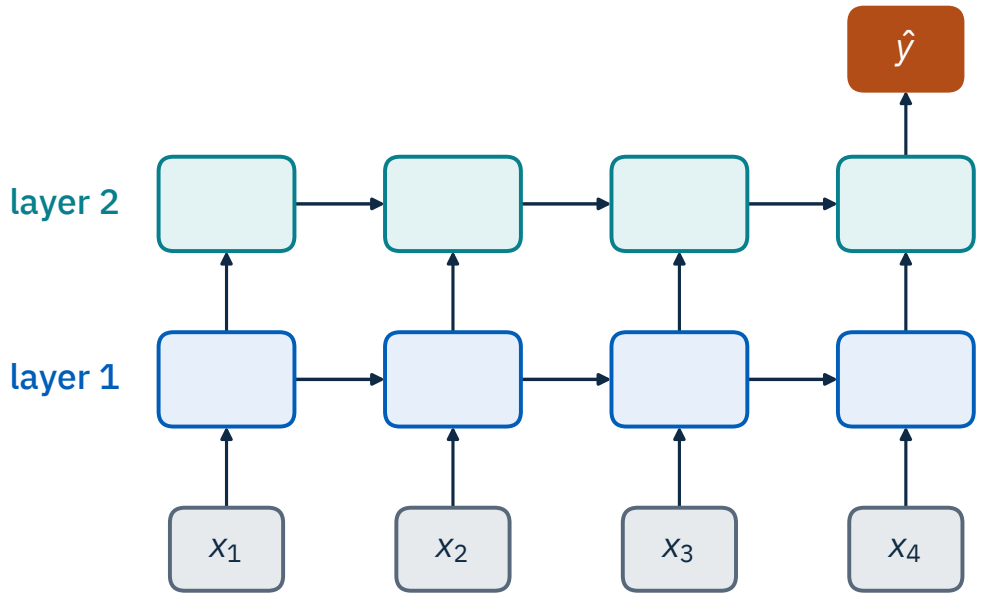
RNN, LSTM and GRU trade simplicity for memory

Model	State	Parameters (14 inputs, 64 units)	Main trade-off
Vanilla RNN	Hidden state	5,120	Simple; long-range training is hard
LSTM	Hidden and cell states	20,480	More gates and parameters
GRU	One gated hidden state	15,360	A simpler gated alternative
1D CNN or TCN	Local feature maps	Depends on kernels and layers	Parallel over time

Compare them with the same split, target and tuning budget.

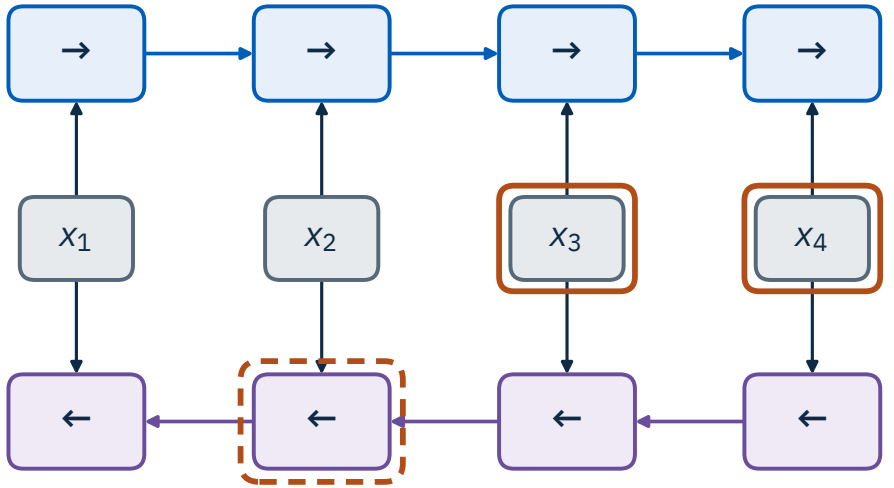
Stacked layers add depth; bidirectional layers read ahead

Stacked: two recurrent layers



Bidirectional: two directions

each output concatenates both directions



the backward state at step 2 has already read x_3 and x_4

Schematic. Left: two stacked recurrent layers, many-to-one. Right: a bidirectional layer; its backward direction runs from the end of the window.

⚠ A bidirectional layer is valid only inside a window that ends at the forecast origin.

A many-to-one LSTM shape check

```python
import torch
from torch import nn

rnn = nn.LSTM(input_size=6, hidden_size=32, batch_first=True)
head = nn.Linear(32, 1)
x = torch.zeros(8, 24, 6)                    # (batch, time, features)
sequence, (hidden, cell) = rnn(x)
prediction = head(sequence[:, -1, :])
print(sequence.shape, hidden.shape, prediction.shape)
```

OUTPUT

```
torch.Size([8, 24, 32]) torch.Size([1, 8, 32]) torch.Size([8, 1])
```

For one layer and equal lengths, `sequence[:, -1]` equals `hidden[0]`.

Hidden state handling is part of the data model

Reset between windows

- The default for independent, shuffled windows
- Every window starts from zeros

Carry state only in order

- Same asset, chronological batches
- Detach at truncation boundaries

Mask padding

- Padded steps are not measurements
- Use packed sequences or masks

Decide the warm-up

- Cold start or a burn-in history
- Identical in training and inference

PRACTICE

5 min · Pairs

Check the lab model before training it

The climate notebook uses `nn.LSTM(14, 128, num_layers=2, batch_first=True)` and a head 128 → 64 → 1.

1. What are the shapes of the LSTM output and of `h_n` for 256 windows of 72 hours?
2. How many parameters does the first LSTM layer have?
3. Which tensor feeds the head, and why not `h_n[0]`?

LABORATORY

60 min

Lab 1A: a warm-up forecast and two baselines

- Shampoo: build 6-month windows, train the LSTM, compare with the previous month.
- Climate: load Jena, subsample hourly, split 70/20/10 in time.
- Fit the scaler on training rows; build 72 h → +12 h windows per split.
- Score persistence and the 24-hour seasonal naive on validation, in °C.
- Start the LSTM training before lunch if time allows.

NOTEBOOK

`exercises/01_lstm_shampoo/LSTM_Shampoo_Student.ipynb`

EVIDENCE TO SUBMIT

- Shampoo RMSE next to the naive RMSE
- Window counts for each split
- Two baseline MAEs on validation

LABORATORY

45 min

Lab 1B: the climate LSTM against both baselines

- [ ] Train the multi-feature LSTM with early stopping; plot both loss curves.
- [ ] Score the test windows in °C: LSTM, persistence and seasonal naive.
- [ ] Use identical target timestamps for all three methods.
- [ ] Plot one test week and name one failure period with a hypothesis.
- [ ] Optional: compare a temperature-only model on validation.

NOTEBOOK

`exercises/02_lstm_climate/LSTM_Climate_Student.ipynb`

EVIDENCE TO SUBMIT

- A score table with both baselines first
- Loss curves with the chosen epoch marked
- One failure period and a hypothesis

DISCUSSION

At 12 hours ahead, persistence is the weak baseline

Jena test windows of the notebook split (6,927 origins); the target is 12 hours after the origin.

4.78 °C

persistence MAE

2.38 °C

24-hour seasonal naive MAE

7.81 °C

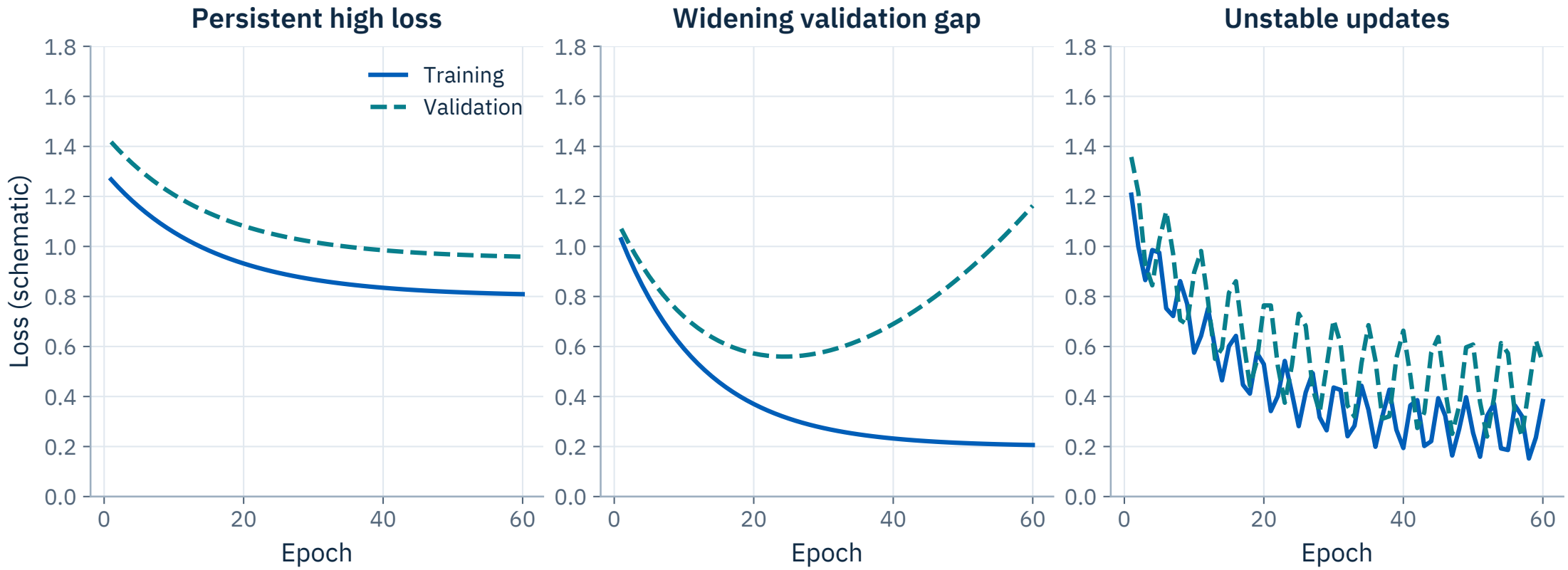
standard deviation of the target

- Report your LSTM next to both; the fair reference is the stronger one.

DISCUSSION

Your loss curves suggest the next experiment

Which shape is closest to your climate LSTM, and what would you change first?



Schematic learning curves (recap of Day 2), not measured training runs; each shape admits several diagnoses.

DISCUSSION

Six errors that flatter a sequence model

✕ Scaler fitted on all rows

- Test statistics leak into training

✕ Windows cross the split

- Training targets inside validation

✕ Different target timestamps

- Baseline and model scored on different rows

Wrong horizon index

- Off by one between origin and target

Test set used for stopping

- The test score is no longer a test

✕ Windows split at random

- Near-copies of test windows in training

PART 4 OF 7

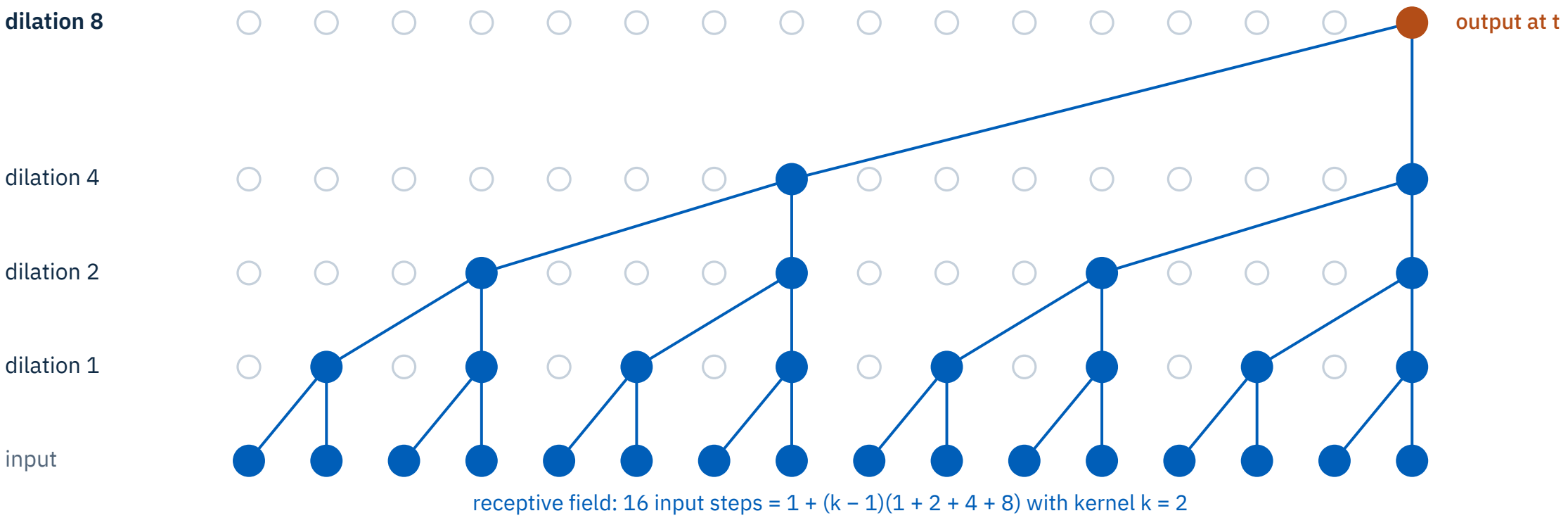
04

Temporal convolutions and attention

Two ways to see far back without a long recurrent chain.

- Dilated causal convolutions
- The encoder–decoder bottleneck
- Queries, keys, values and masks
- Multi-head attention

Dilated causal convolutions reach far back cheaply



Schematic of a causal temporal convolutional stack (kernel size 2, dilations 1, 2, 4, 8). Highlighted: every input that can influence the last output.

Receptive field grows with kernel size and dilation

$$R = 1 + (k - 1)\sum_{l} d_l$$

16

k = 2, dilations 1, 2, 4, 8

31

k = 3, dilations 1, 2, 4, 8

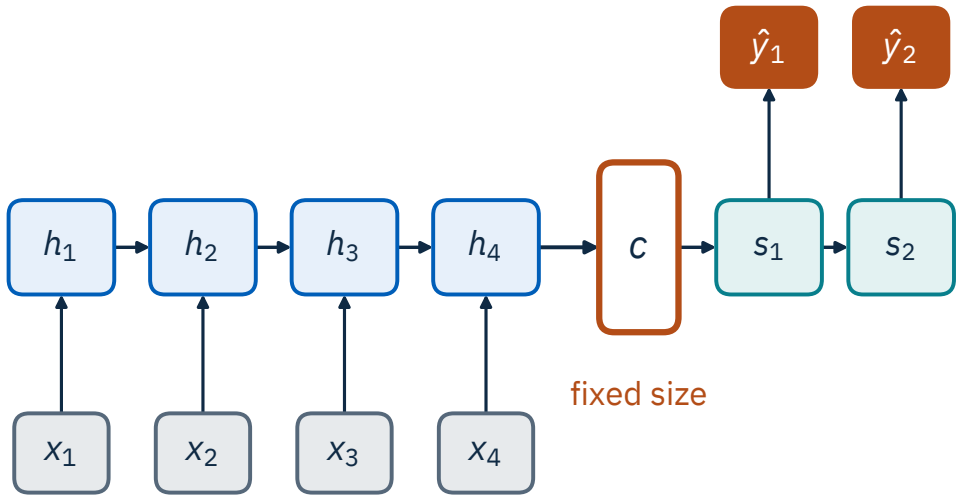
9

k = 3, four layers, no dilation

- One week of hourly data (168 steps) needs 8 doubling layers with k = 2: R = 256.

One context vector is a bottleneck

Without attention: one context vector

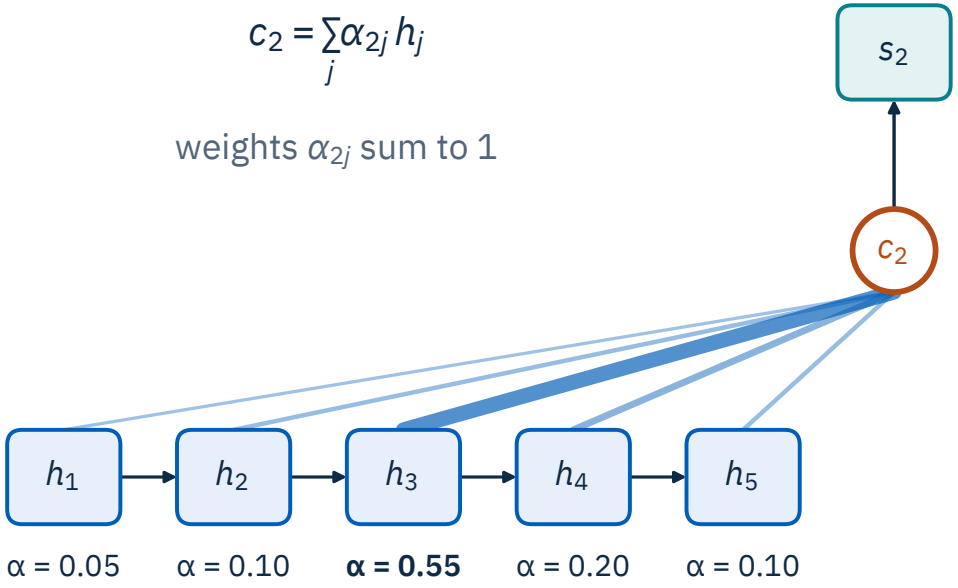


every output step sees only c

With attention: a new context for each output

$$c_2 = \sum_j \alpha_{2j} h_j$$

weights α_{2j} sum to 1

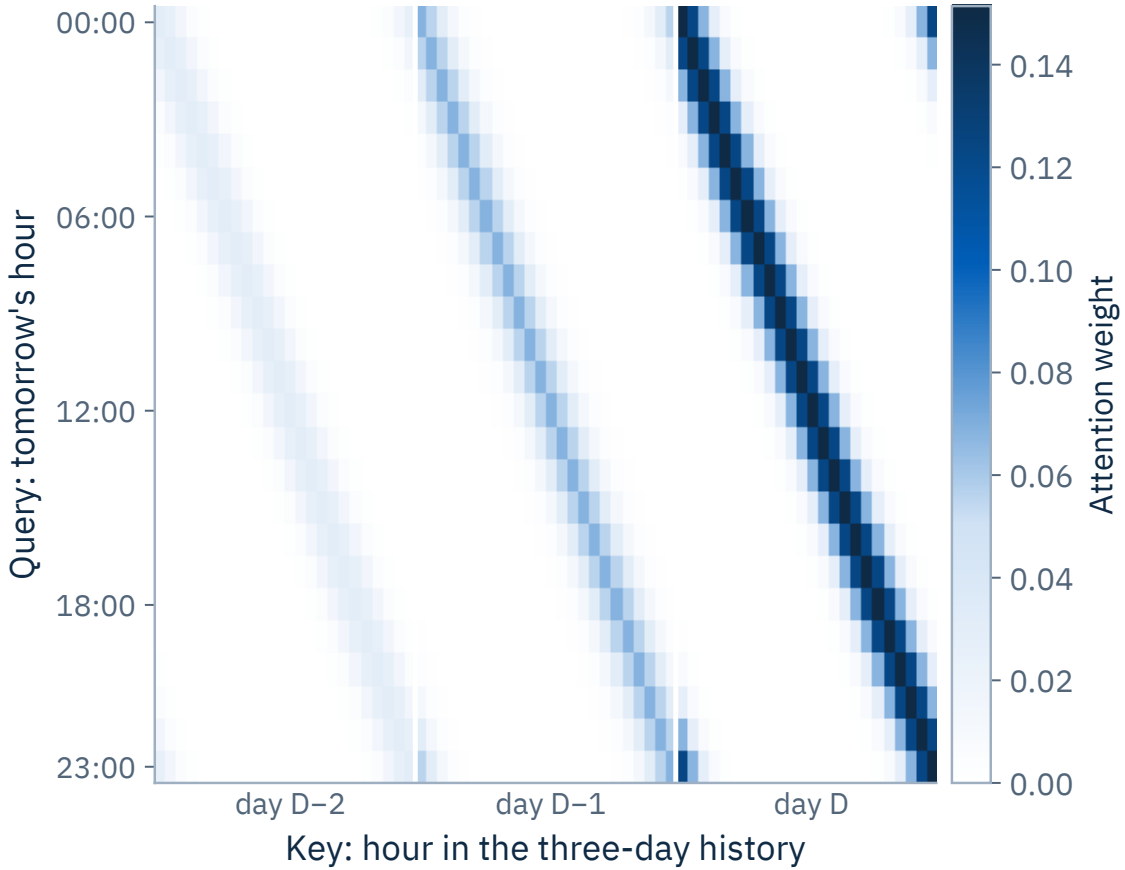


line width shows the attention weight for output step 2

Schematic. Left: an encoder–decoder that passes one fixed-size vector. Right: attention forms a weighted context for each output step; weights are illustrative.

Attention is a learned soft alignment

- Each row is a probability distribution over the keys.
- Each hour of tomorrow looks at the same clock hour.
- Recent days weigh more: day D gets 61% at 00:00.



Hand-built queries and keys (clock-hour encodings, recency penalty); illustrates the mechanism, not a trained model.

◆ Attention weights show where information flowed, not why the output is right.

Queries, keys and values have distinct roles

Quantity	Role	In self-attention
Query Q	What each position seeks	A projection of the input states
Key K	What each position offers for matching	Another projection of the same states
Value V	Information to aggregate	A third projection of the same states

Cross-attention takes queries from one sequence and keys and values from another.

◆ Each output is a sum of values, weighted by how well its query matches each key.

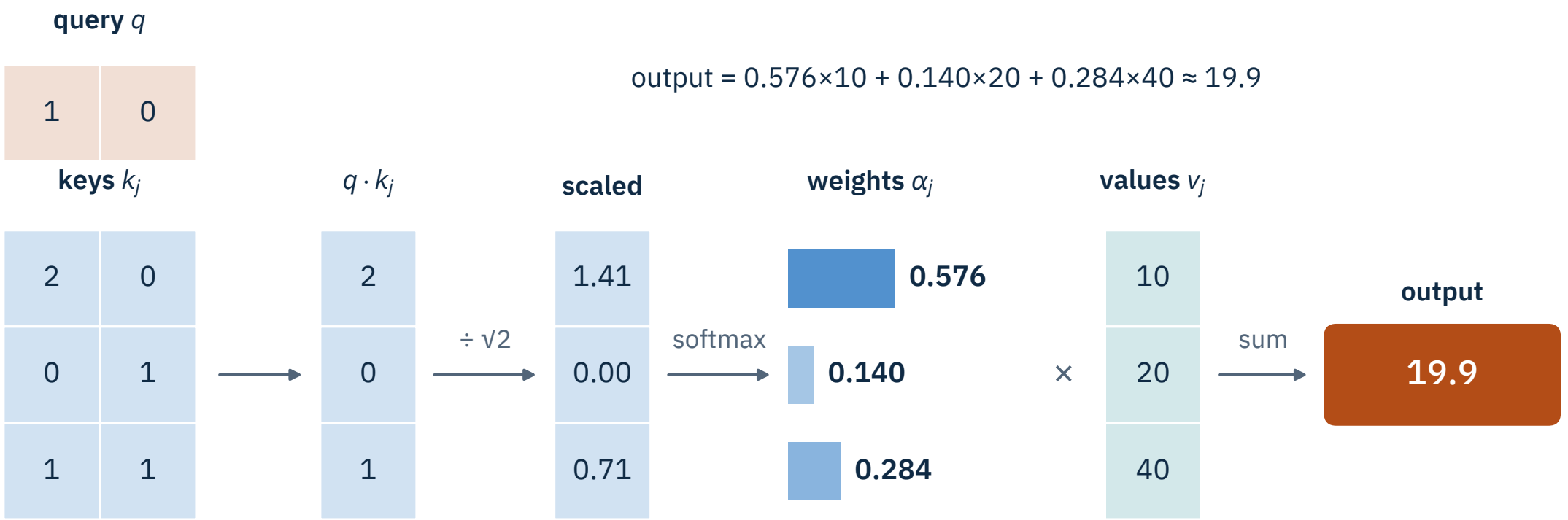
Scaled dot-product attention in one line

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

- Divide by $\sqrt{d_k}$ to keep scores in a useful range.
- Softmax runs over the keys, separately for each query.
- The mask M adds $-\infty$ where access is not allowed.
- Each output is a weighted average of value vectors.

◆ PyTorch: `F.scaled_dot_product_attention(q, k, v, is_causal=True)`.

Attention for one query, step by step



Exact arithmetic for one query, three keys and scalar values; $d_k = 2$.

PRACTICE

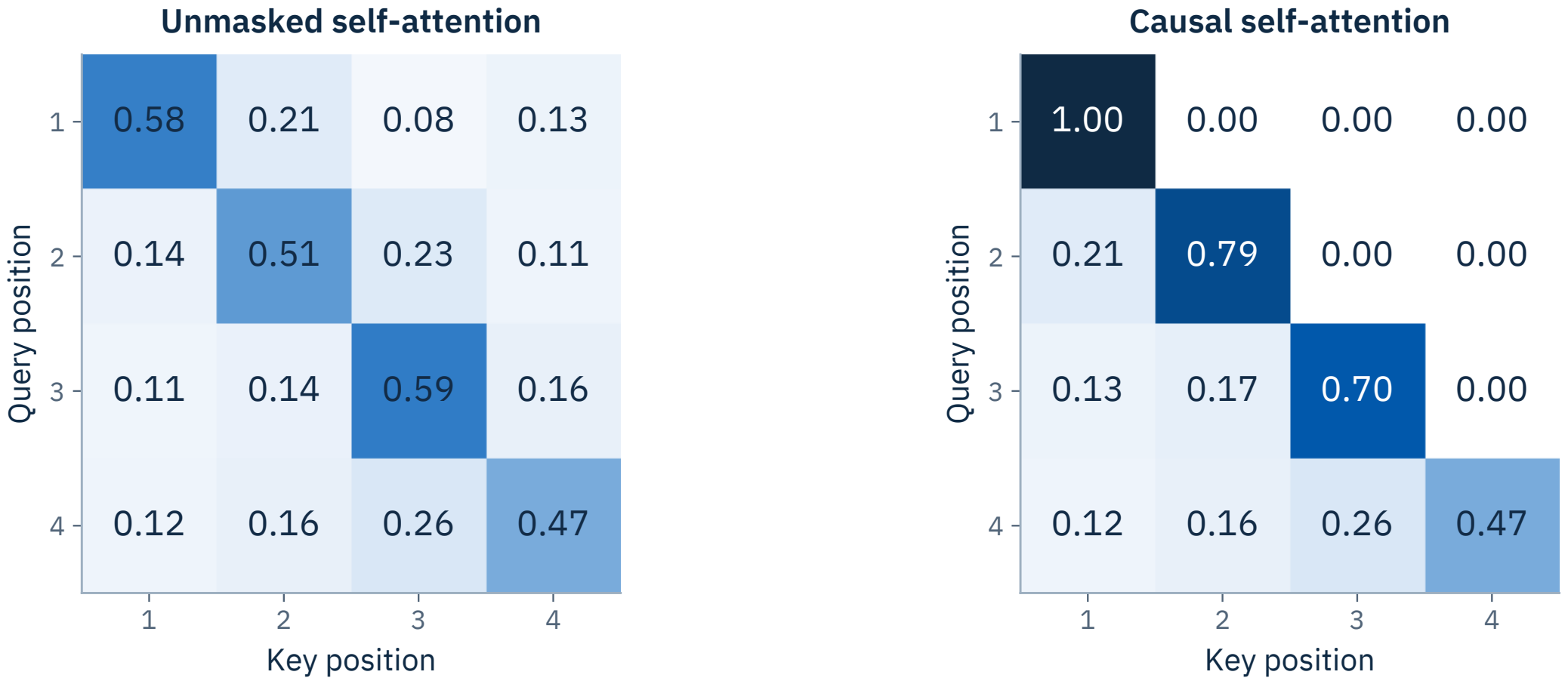
4 min Individual

One attention calculation

A query has scaled scores [0, log 2] for two keys. The values are [10, 20].

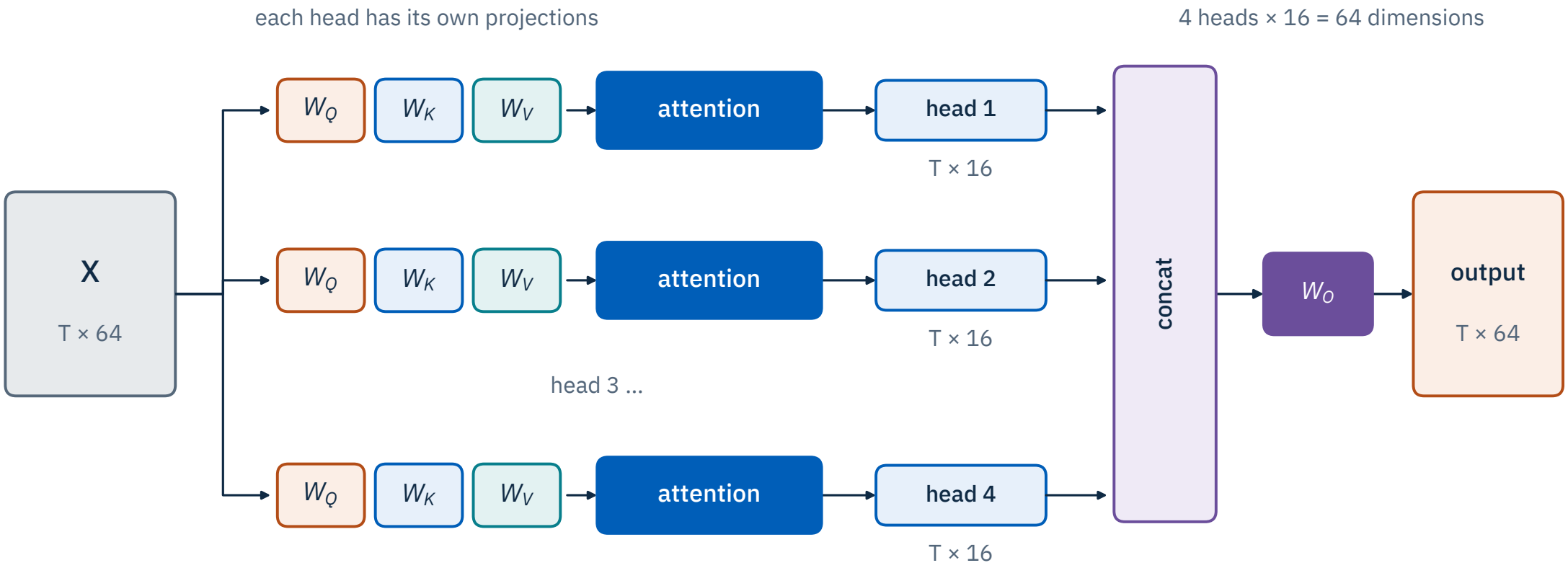
1. Calculate the softmax weights.
2. Calculate the attention output.
3. What changes if the second key is masked out?

A causal mask removes future access



Computed example attention weights with and without an additive causal mask. Every row normalizes over its allowed keys.

Multiple heads attend in different subspaces



Schematic of multi-head self-attention with model width 64 and 4 heads of width 16.

PRACTICE

4 min · Pairs

Why divide by the square root of d_k?

Query and key components are independent, with mean 0 and variance 1; $d_k = 64$.

1. What is the standard deviation of q · k before and after the scaling?
2. Compare softmax([8, 0, −8]) with softmax([1, 0, −1]).
3. Why would near one-hot weights slow down learning?

PART 5 OF 7

05

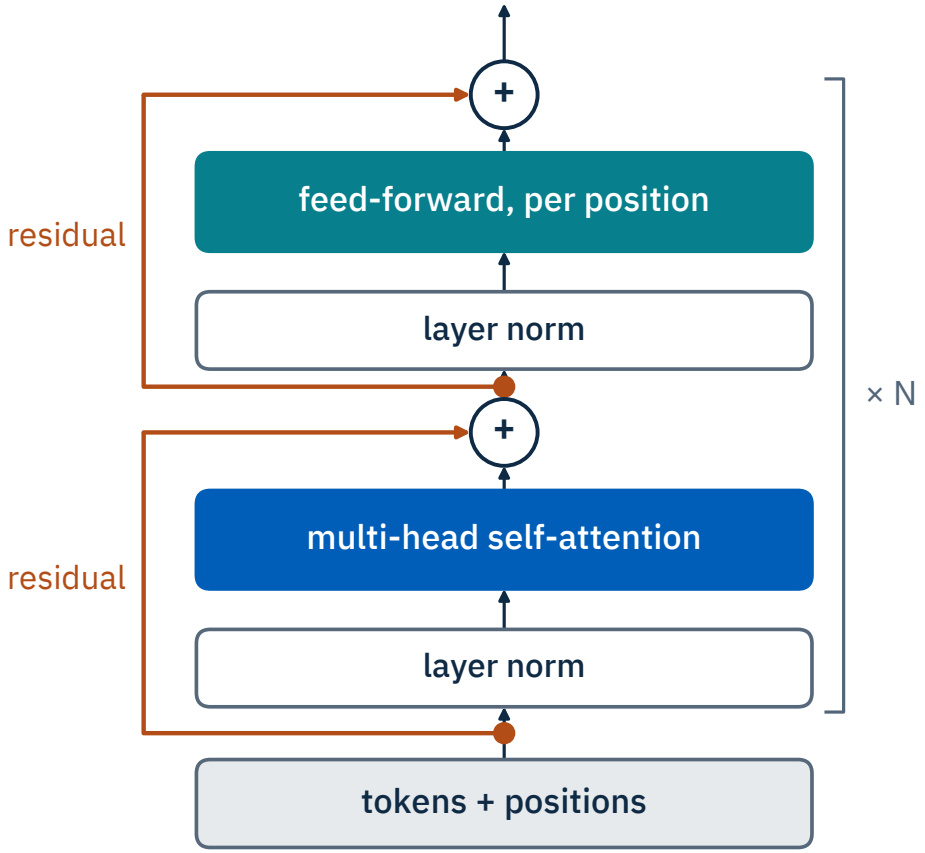
Transformers and pretrained models

Attention blocks, positions, and models trained at scale.

- The transformer block and positional information
- Teacher forcing and the cost of long sequences
- Pretrained forecasters
- Language models in energy workflows

A transformer block alternates mixing and processing

- Self-attention mixes information between positions.
- The feed-forward network transforms each position alone.
- Residual additions keep a direct update path.
- Layer norm controls scale; its placement varies.

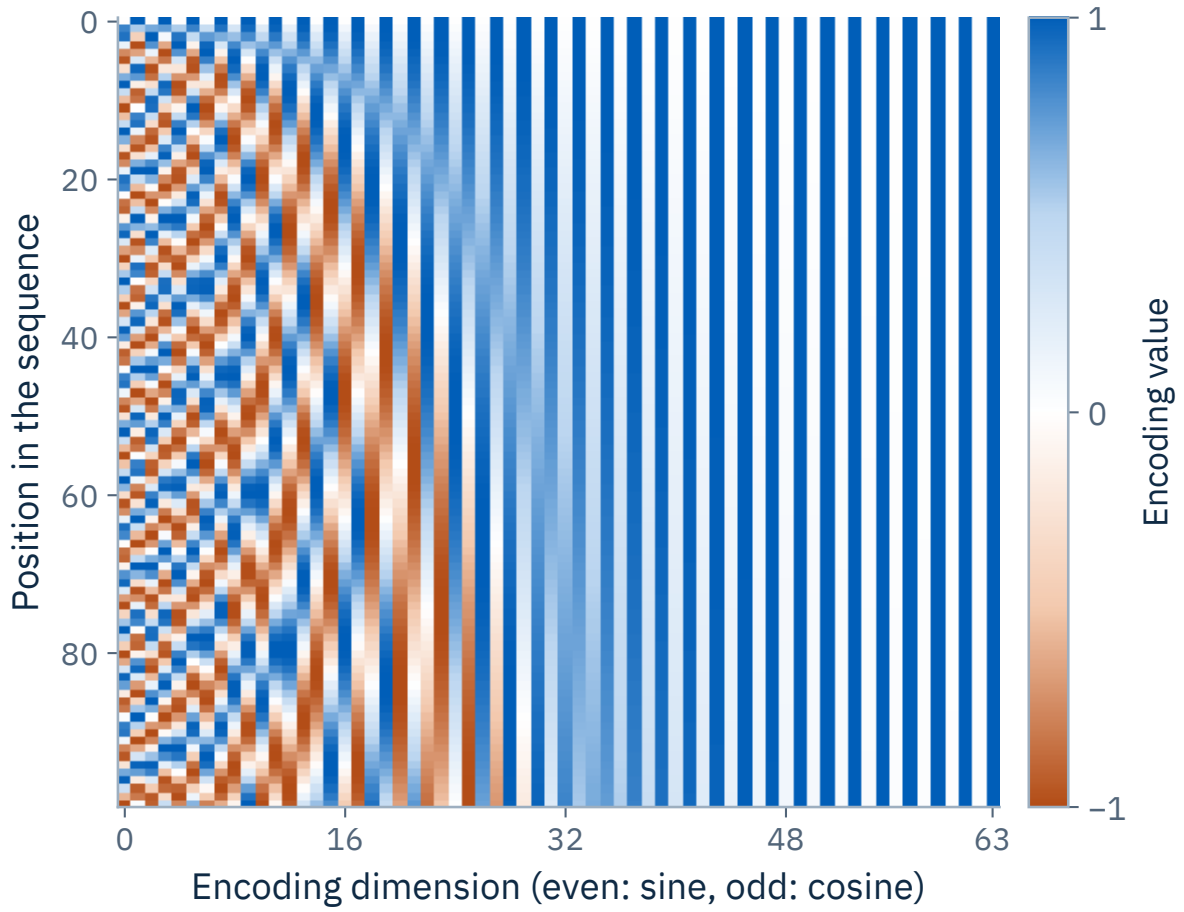


Schematic of a pre-norm encoder block; the 2017 original applied layer norm after each residual addition.

Position must enter the representation

$$PE_{p,\,2i} = \sin\left(p \,/\, 10000^{2i/d}\right)$$
$$PE_{p,\,2i+1} = \cos\left(p \,/\, 10000^{2i/d}\right)$$

- Self-attention alone ignores the order of positions.
- Options: sinusoidal, learned, relative, rotary.
- Time series also need elapsed time and calendar.



Exact sinusoidal encoding for model width 64 and positions 0–99.

Transformer families match different tasks

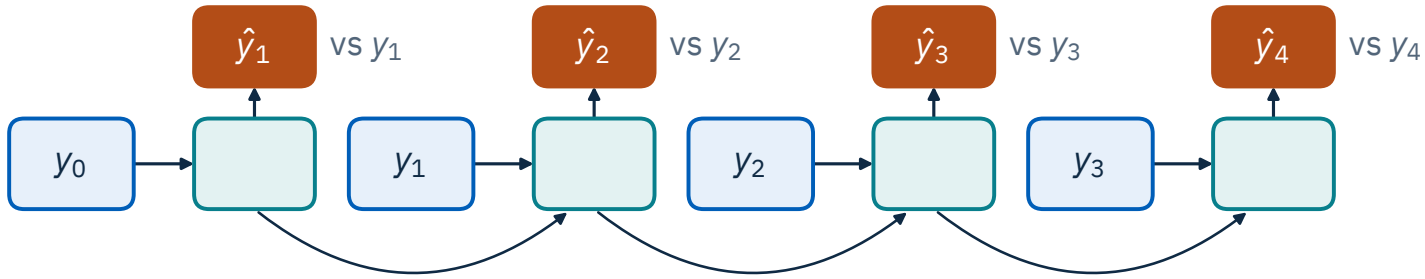
Family	Attention pattern	Example use
Encoder	Often bidirectional over available input	Representation or classification
Decoder-only	Causal over generated positions	Next-token prediction
Encoder–decoder	Input encoding plus cross-attention	Translation or sequence generation

All three can forecast if the masks respect the forecast origin.

- Many large language models are decoder-only transformers trained to predict the next token.

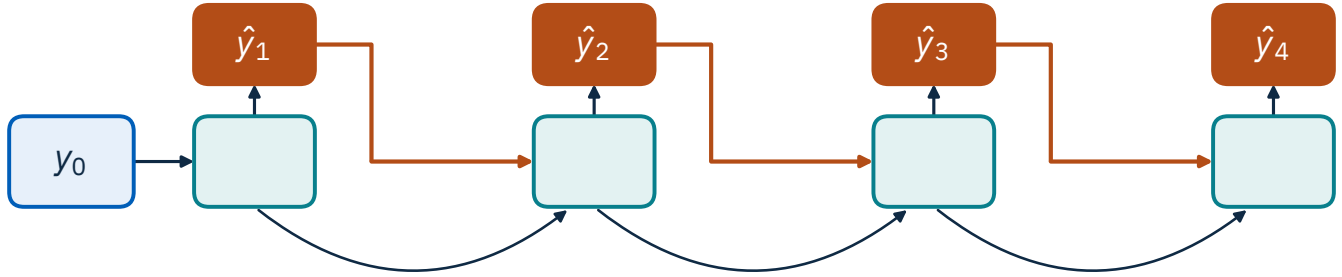
Teacher forcing and inference see different histories

Training with teacher forcing: each step reads the true previous value



loss at every step;
inputs are observed values

Inference, autoregressive: each step reads its own previous forecast

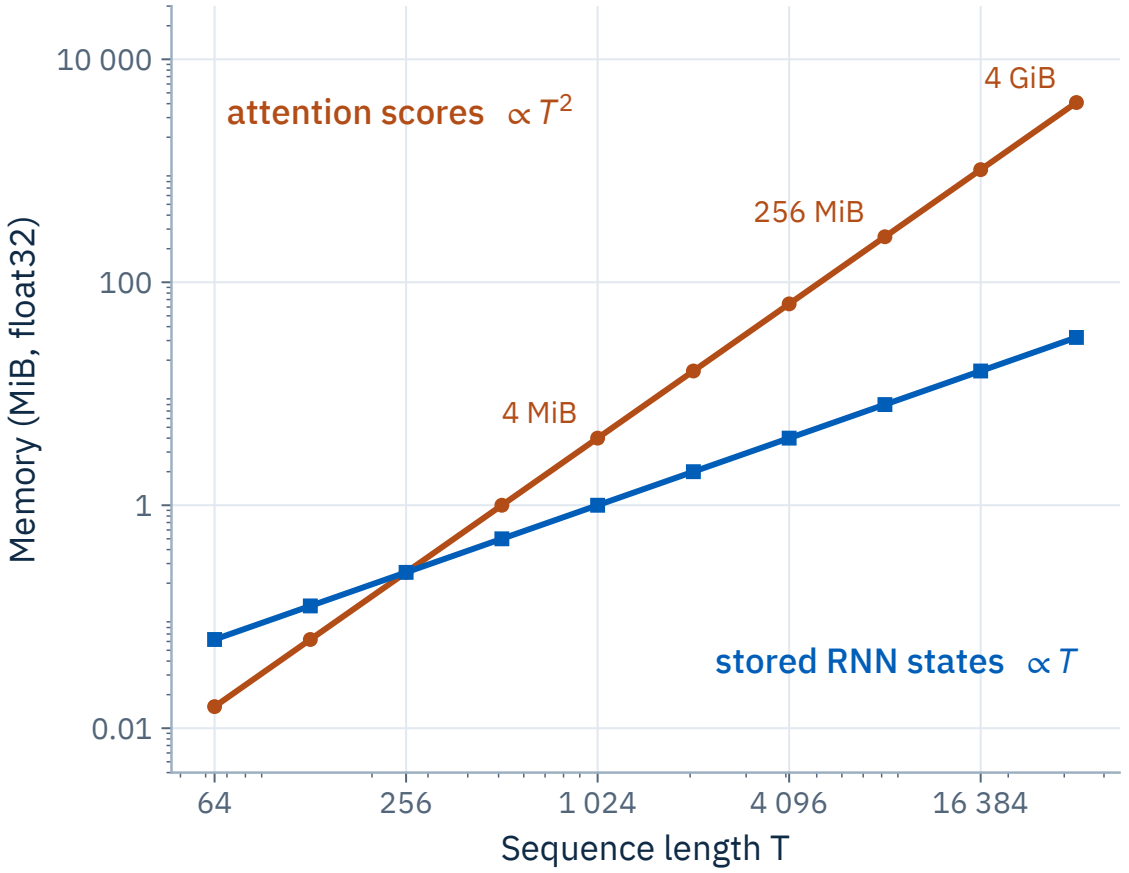


forecast errors are
fed back as inputs

Schematic of a recurrent decoder in training (teacher forcing) and in inference (autoregressive rollout); y_0 is the last observed value.

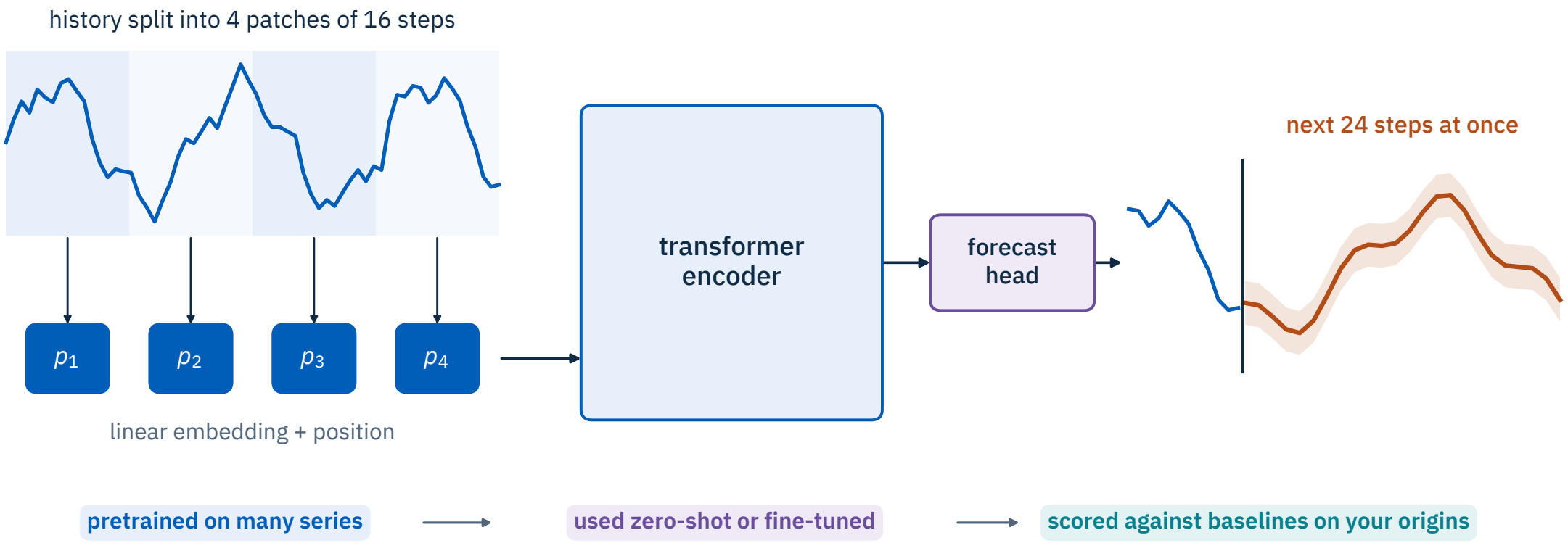
Dense attention cost grows with the square of length

- The score matrix has T^2 entries per head.
- One year of hourly data: T = 8,760, about 293 MiB per head.
- Efficient kernels cut memory traffic; compute stays quadratic.
- Measure memory and latency at your real input lengths.



Exact arithmetic for float32 scores, one head and one sequence, against stored RNN states of width 256.

Time-series transformers turn patches into tokens



Schematic of a patch-based transformer forecaster; the curves are a synthetic illustration, not model output.

Pretrained forecasters still face your baselines

What they are

- Trained on many public series
- Used zero-shot or fine-tuned
- Some accept covariates, some do not

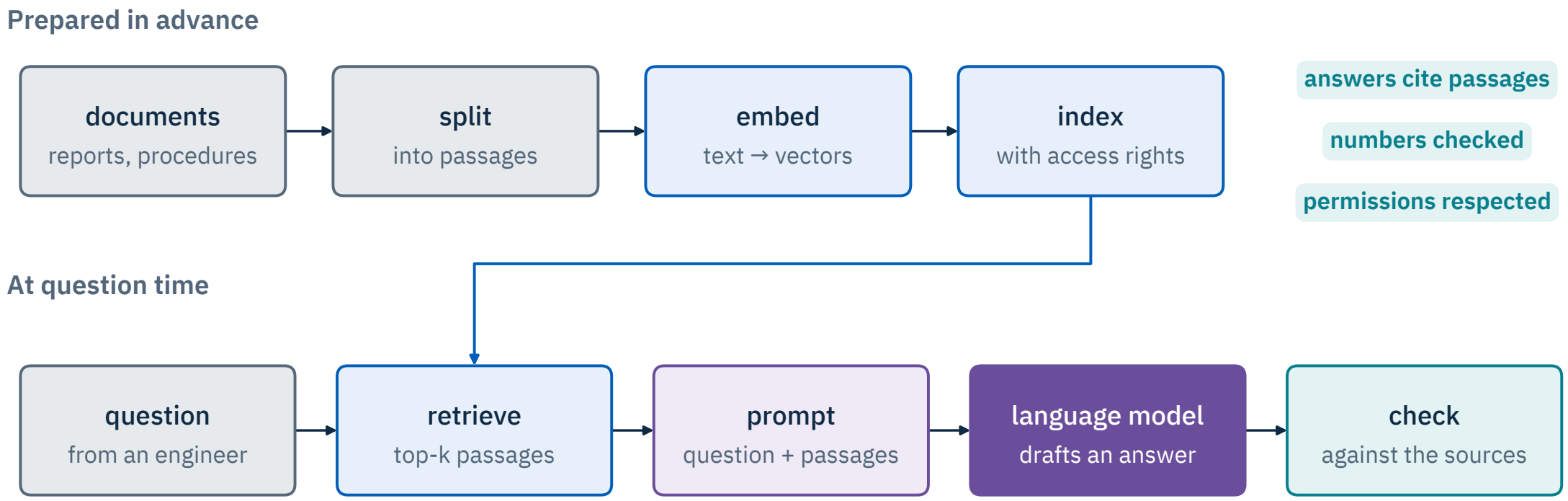
What to check

- Frequency, horizon and context length
- Overlap with the pretraining data
- Probabilistic output: is it calibrated?

How to judge

- Same rolling origins as baselines
- Error by horizon, cost and latency
- Licence and data governance

Language models can search documents and draft code



Schematic of retrieval-augmented generation for technical documents.

Generated answers need checking against sources

✓ Useful with checks

- Search and summarize reports, with citations
- Draft analysis code, then test it
- Extract fields from forms for review

vs

✗ Risky without checks

- Fluent but wrong numbers or units
- Confidential data sent to outside services
- Decisions without domain review

◆ Evaluate a retrieval system on source relevance and answer faithfulness.

PRACTICE

5 min · Pairs

Match the architecture to the constraint

Choose a starting point after the baselines, and name one risk.

1. ESP sensors every 10 minutes; a real-time alarm from the last 2 hours.
2. Hourly load for 300 feeders; one-week context; day-ahead horizon.
3. 5,000 well reports; "which wells had stuck pipe after a mud change?"

PART 6 OF 7

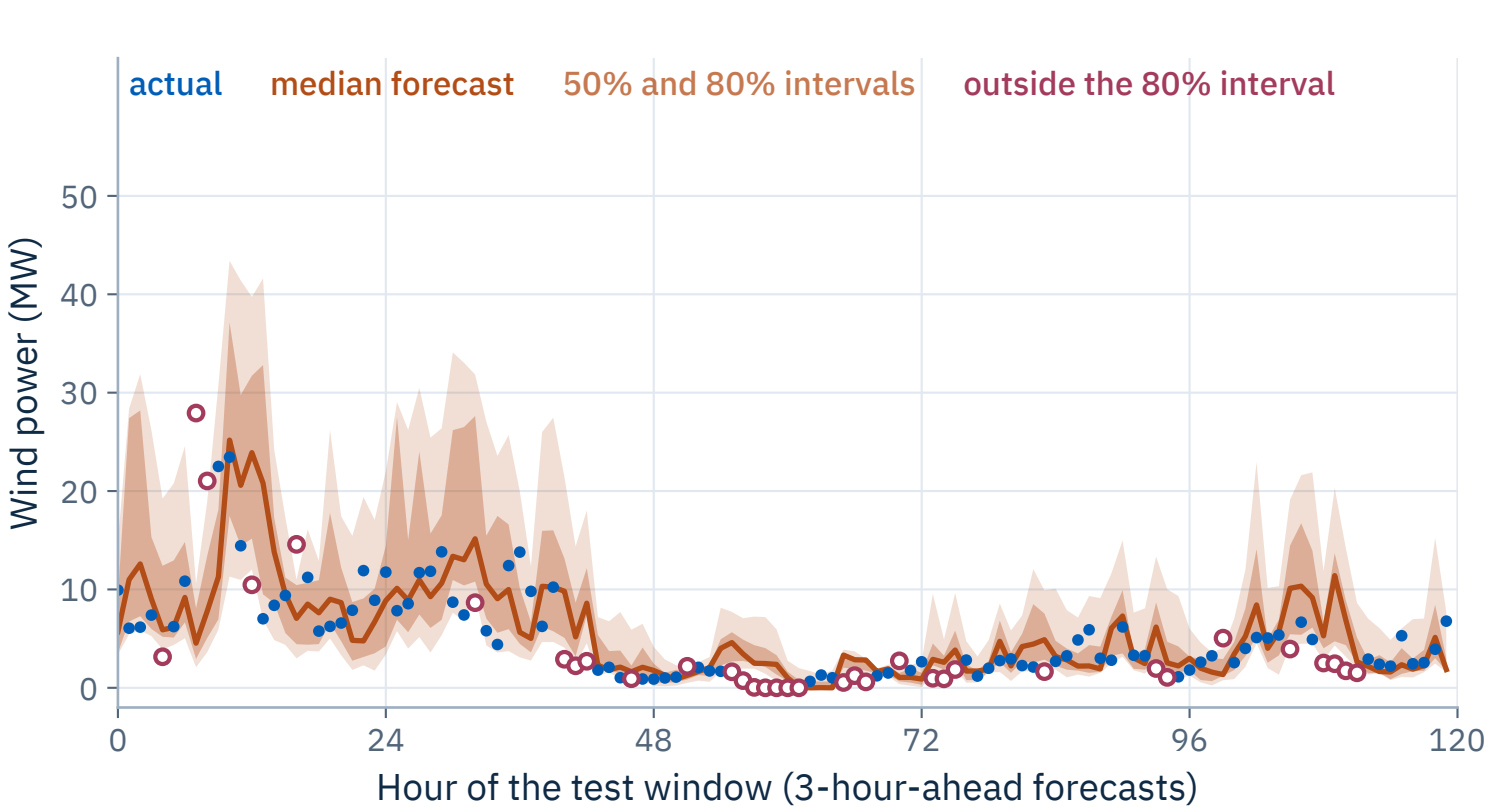
06

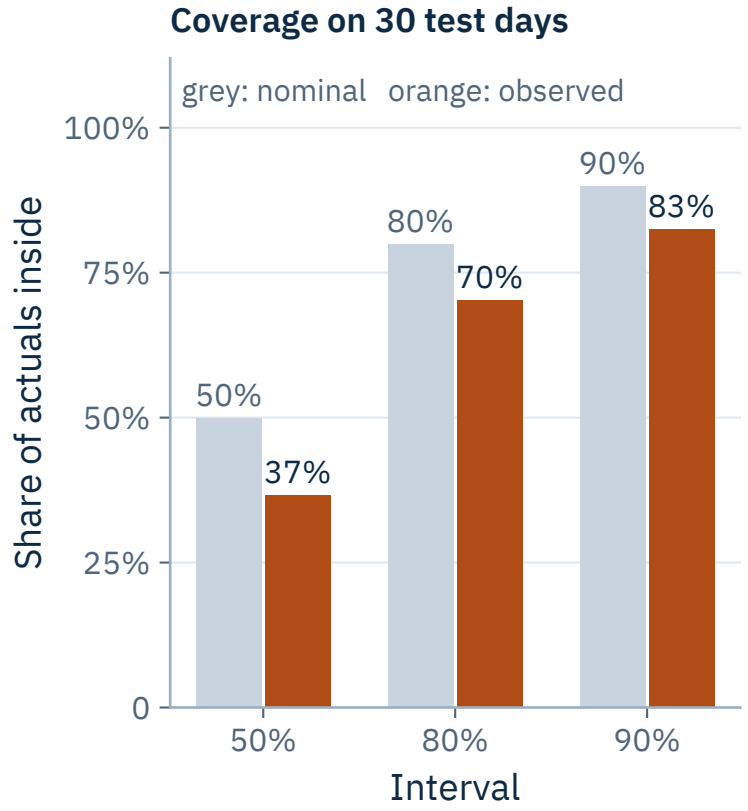
Uncertainty and sequence anomalies

How wrong might the forecast be, and is the process normal?

- Quantile forecasts and the pinball loss
- Prediction intervals and their coverage
- LSTM autoencoders for anomaly detection
- Thresholds, events and applications

A forecast should state its uncertainty



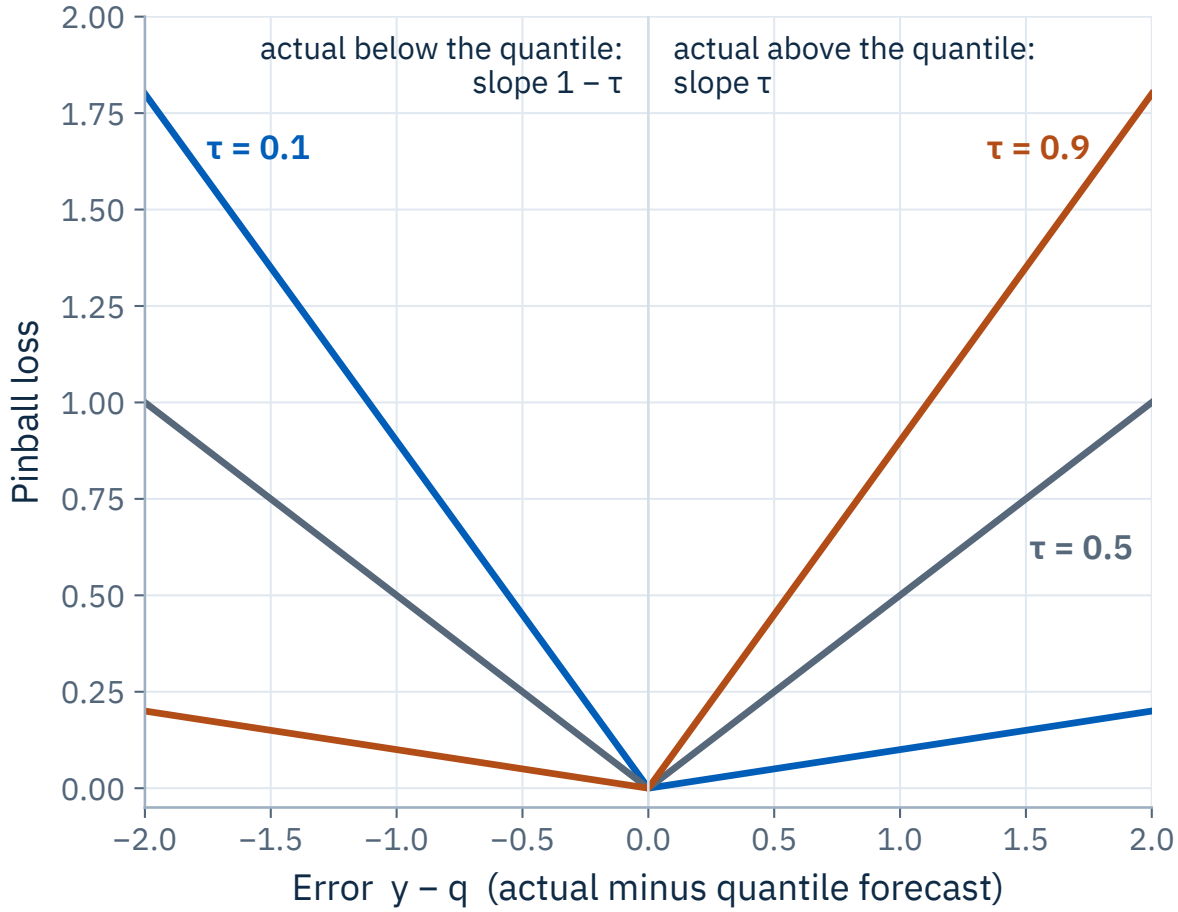


Synthetic 50 MW wind farm (seed 3). Quantile gradient boosting, 3 hours ahead, on the last six hourly values and hour of day; trained on days 1–90, scored on days 91–120.

Quantile forecasts minimize the pinball loss

$$L_\tau(y, q) = \max(\tau(y - q),\ (\tau - 1)(y - q))$$

- τ = 0.9 penalizes under-forecasts nine times more than over-forecasts.
- The minimizer is the τ-quantile of the outcome.
- One output per quantile, trained with one loss each.



Exact quantile (pinball) loss for τ = 0.1, 0.5 and 0.9.

PRACTICE

4 min · Individual

Pinball loss and coverage by hand

1. For $\tau = 0.9$ and an actual of 100 MW, compute the loss for $q = 90$ and for $q = 110$.
2. A nominal 80% interval contains 150 of 200 test hours. What is its coverage?
3. Is the interval too narrow or too wide, and what would you do?

Judge intervals by calibration, then sharpness

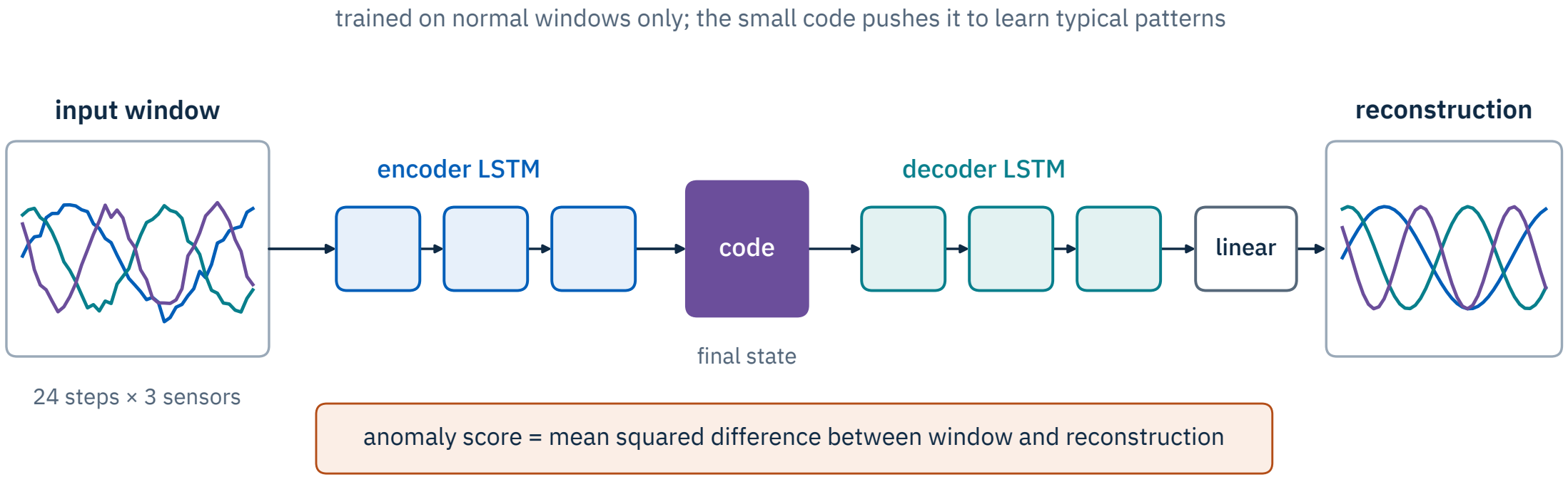
A probabilistic forecast is evaluated in a fixed order, on held-out periods only.

1. **Coverage** — share of actuals inside each interval
2. **Width** — narrower is better, at correct coverage
3. **Pinball loss** — averaged over the quantiles
4. **Breakdown** — by horizon, season and regime
5. **Recalibrate** — on a recent validation period

◆ An interval that is sharp but under-covers promises more certainty than it has.

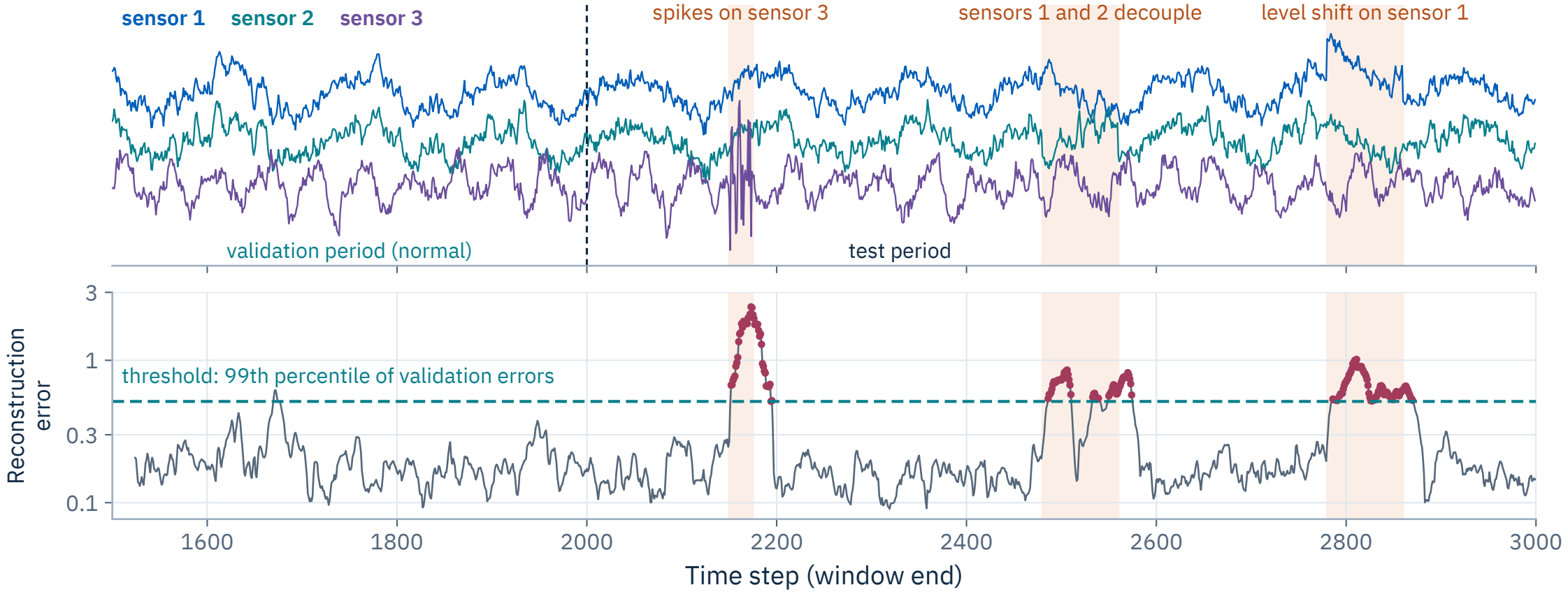
An LSTM autoencoder learns to reconstruct normal windows

Train on normal operation only; windows that differ from normal reconstruct badly.



Schematic (curves illustrative). Notebook 04 uses 30-step windows of the sensor columns.

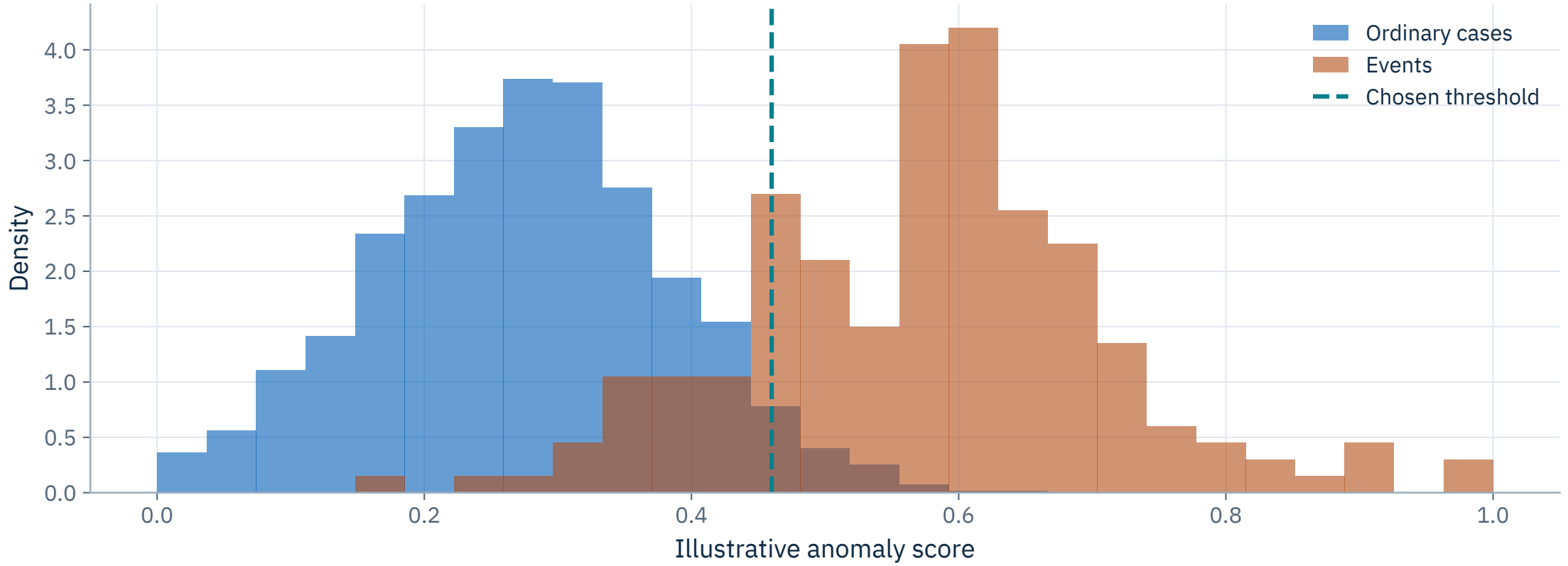
Reconstruction error flags unusual windows



Synthetic three-sensor series (seed 5) with three injected anomalies; LSTM autoencoder (16 units, 24-step windows) trained on steps 0–1,499 (torch seed 0).

Any threshold trades missed events for false alarms

Moving the line right removes false alarms and misses more events; no position removes both.



Simulated score distributions (seed 42), not a fitted detector or calibrated probabilities. The machine learning course uses the same example.

Thresholds and events decide what an alert means

Fit on past normal data

- Scaler and threshold from earlier periods
- Never from errors on training windows

Score events, not windows

- Overlapping windows repeat one event
- Report lead time and false alarms per week

Expect drift

- New regimes look anomalous
- Retrain and re-threshold on a schedule

◆ Lab 2 applies these rules to real paper-machine data.

Each application needs its own target and evaluation

Application	Possible target	Evaluation priority
ESP monitoring	Future event within a defined horizon	Event recall, lead time, false alerts
Production forecasting	Rate at one or more future horizons	Error by well and horizon
Drilling ROP	Future rate of penetration	Covariates available at the origin
Load and renewables	Hourly values, day ahead	Error by horizon, interval coverage
Sequence autoencoder	Reconstruction or prediction residual	Threshold validation and regime effects

These are task definitions to evaluate, not established capabilities.

LABORATORY

35 min

Lab 2: sequence anomalies or a retail forecast

- Option A: train the LSTM autoencoder on an early normal period only.
- Set the threshold on a later normal period; score the remaining rows.
- Report events found, events missed and false alarms, not only F1.
- Option B: retail forecast for one item; beat the weekly seasonal naive.
- Either option: one plot, one table and one caveat.

NOTEBOOK

`exercises/04_timeseries_autoencoder/TS_Autoencoder_Student.ipynb`

EVIDENCE TO SUBMIT

- Error-over-time plot with the threshold
- Event or baseline comparison table
- One caveat about the evaluation

DISCUSSION

What did the alerts, or the forecasts, really show?

- How many distinct events were detected, and how early?
- How many alerts per week would operators receive?
- Did the threshold come from data the model never trained on?
- Retail: did the LSTM beat the weekly seasonal naive on MAE?

◆ Report events and alarms per week; an F1 score on rows hides both.

PART 7 OF 7

07

Course review

Five days, one habit: evidence before architecture.

- A map of the five days
- Choosing a model family
- The final review exercise
- Key takeaways and exit check

COURSE REVIEW

Five days build one chain of evidence

Day 1
Neural network foundations

- neurons and MLPs
- losses, gradients
- backpropagation

Day 2
Training neural networks reliably

- initialization
- regularization
- optimizers, tuning

Day 3
Energy modelling capstone

- data contracts
- tabular-NN playbook
- error analysis

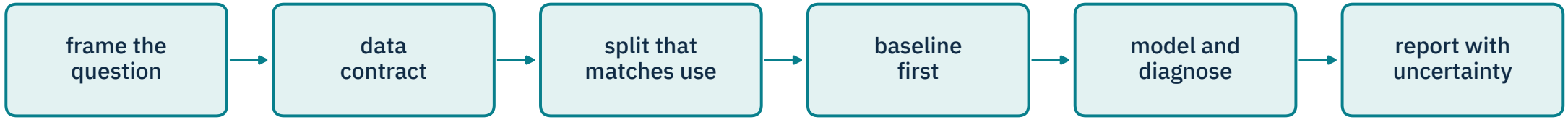
Day 4
Convolutional neural networks

- convolution, pooling
- transfer learning
- 1D CNNs, imaging

Day 5
Sequence models and attention

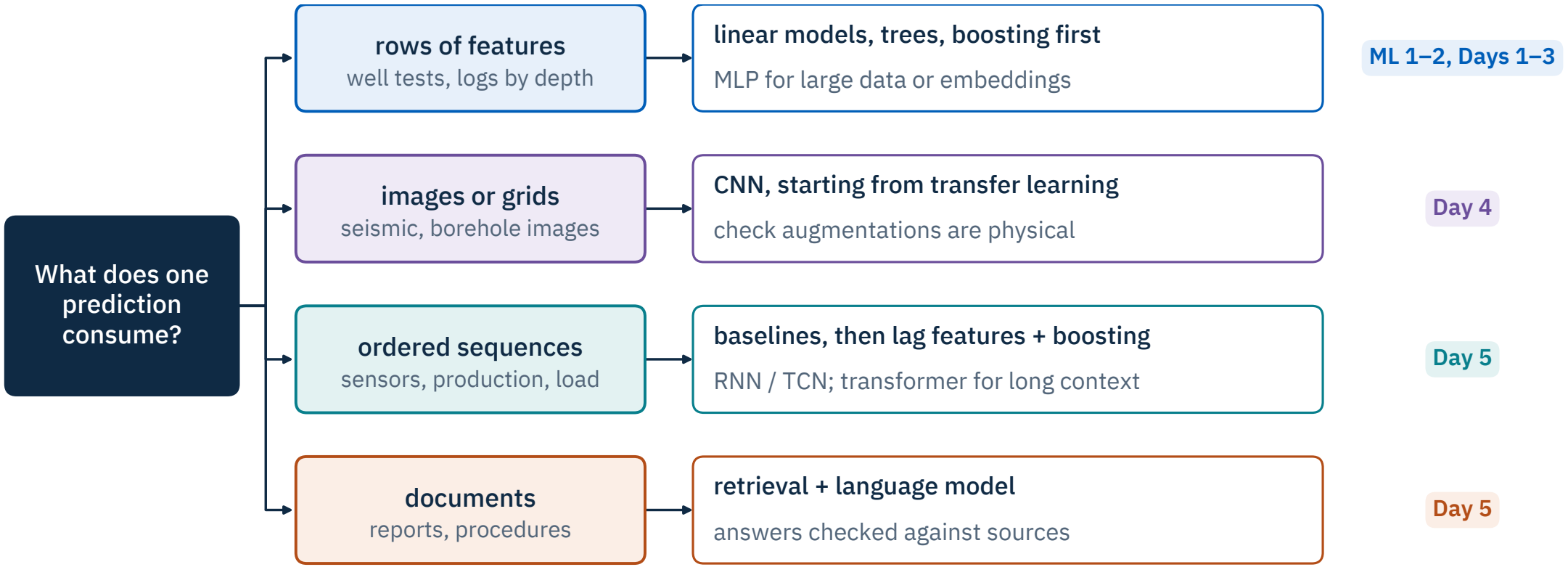
- windows, baselines
- RNN, LSTM, TCN
- attention, intervals

the evidence chain used every day



Schematic summary of the five days of the neural networks course.

Choose the model family from the data geometry



Schematic model-choice guide: a starting point to test against baselines, not a rule.

Every model family shares the same evaluation habits

Data	Split that matches use	First baseline	Report
Rows per well or asset	Grouped by well, often also by time	Mean or linear model	Error by well
Images or grids	By survey, well or field	Pretrained features + linear	Recall per class
Sequences	Rolling origins in time	Persistence, seasonal naive	Error by horizon
Documents	By document set	Keyword search	Faithfulness to sources

Uncertainty and monitoring apply to all four.

PRACTICE

10 min · Pairs

Defend a model choice for one energy task

Pick one: ESP gas-lock warning 1 hour ahead; facies from well-log curves; day-ahead feeder load.

1. Choose a dense model, CNN, RNN or attention model and defend the choice.
2. Define the tensor shapes, target timing, baseline and evaluation split.
3. Explain one plausible failure and the evidence that would reveal it.

DISCUSSION

A strong answer names the timing before the network

1. **Unit and timing**: what one prediction consumes, and when it is made
2. **Shapes**: (batch, time, features) in; the target shape out
3. **Baseline**: the simplest method with exactly the same timing
4. **Split**: grouped or rolling origin, matching deployment
5. **Failure evidence**: the breakdown that would expose the weakness

Key takeaways

- ✓ Every forecast is defined by its origin, horizon and the inputs available then.
- ✓ Beat persistence and seasonal naive on identical targets before claiming skill.
- ✓ Gates, dilations and attention give information and gradients shorter paths.
- ✓ Report errors by horizon, with intervals whose coverage you have checked.
- ✓ Choose the architecture from the data geometry; evaluate every one the same way.

PRACTICE

4 min · Individual

Exit check: five quick questions

1. A forecast issued at 10:00 targets 18:00 tomorrow. What is the horizon?
2. Which baseline would you compute first for hourly load, and why?
3. Which quantity does the LSTM forget gate multiply?
4. Which positions does a causal mask hide from a query?
5. An 80% interval covers 64% of test hours. What do you conclude?

References and further study

- Hyndman and Athanasopoulos, *Forecasting: Principles and Practice*, 3rd edition (online); Hyndman and Koehler (2006) on MASE.
- Hochreiter and Schmidhuber (1997), long short-term memory; Cho and colleagues (2014), GRU; Bai, Kolter and Koltun (2018), TCN.
- Bahdanau, Cho and Bengio (2014), attention; Vaswani and colleagues (2017), *Attention Is All You Need*; Nie and colleagues (2023), PatchTST.
- Koenker and Bassett (1978), quantile regression; Gneiting and Raftery (2007), proper scoring rules.
- PyTorch documentation (LSTM, GRU, MultiheadAttention) and the Keras weather-forecasting example.